

Although the model performs well (especially in detecting malware), an analysis of the metrics by class reveals significant difficulty in correctly identifying benign samples. The low recall for class 0 suggests that, although API frequency is informative, data imbalance and loss of order prevent the model from generalizing well to the minority class.

1.3 Task 2 - Feed Forward Neural Network (FFNN)

text

```
[ ]: # Create directory for plots
save_dir = results_path + 'images/' + 'task2_plots/'
os.makedirs(save_dir, exist_ok=True)
```

1.3.1 API Call Statistics

```
[ ]: # --- Compute the length (number of API calls) per sequence ---
df_train['seq_length'] = df_train['api_call_sequence'].apply(len)
df_test['seq_length'] = df_test['api_call_sequence'].apply(len)

# --- Basic descriptive statistics ---
print("Train set sequence length stats:")
print(df_train['seq_length'].describe(), "\n")

print("Test set sequence length stats:")
print(df_test['seq_length'].describe(), "\n")

# --- Check if all sequences have the same length ---
same_length_train = df_train['seq_length'].nunique() == 1
same_length_test = df_test['seq_length'].nunique() == 1
print(f"All train sequences same length? {same_length_train}")
print(f"All test sequences same length? {same_length_test}\n")
```

Train set sequence length stats:

count	16325.000000
mean	75.030812
std	8.951411
min	60.000000
25%	67.000000
50%	75.000000
75%	83.000000
max	90.000000

Name: seq_length, dtype: float64

Test set sequence length stats:

count	6505.000000
mean	86.331130
std	9.113511
min	70.000000
25%	79.000000

```

50%      87.000000
75%      94.000000
max      100.000000
Name: seq_length, dtype: float64

```

```

All train sequences same length? False
All test sequences same length? False

```

```
[ ]: df_train
```

```

[ ]:
      api_call_sequence  is_malware  \
0      [LdrGetDllHandle, LdrGetProcedureAddress, LdrL...      1
1      [NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...      1
2      [FindResourceExW, LoadResource, FindResourceEx...      1
3      [FindResourceExW, LoadResource, FindResourceEx...      1
4      [LdrGetProcedureAddress, SetErrorMode, LdrLoad...      1
...
16320  [LdrGetProcedureAddress, LdrLoadDll, LdrGetPro...      1
16321  [NtClose, LdrGetProcedureAddress, CryptCreateH...      1
16322  [LdrGetProcedureAddress, LdrGetDllHandle, LdrG...      1
16323  [LdrGetProcedureAddress, LdrGetDllHandle, LdrG...      1
16324  [SetFilePointer, NtReadFile, SetFilePointer, N...      1

```

```

      seq_length
0              73
1              88
2              79
3              71
4              63
...
16320          64
16321          78
16322          84
16323          62
16324          77

```

```
[16325 rows x 3 columns]
```

```

[ ]: # --- Compare distributions visually ---

plt.figure(figsize=(6, 4))
plt.hist(df_train['seq_length'], bins=50, alpha=0.6, label='Train',
         density=True)
plt.hist(df_test['seq_length'], bins=50, alpha=0.6, label='Test', density=True)
plt.xlabel("Number of API calls per sequence")
plt.ylabel("Density")

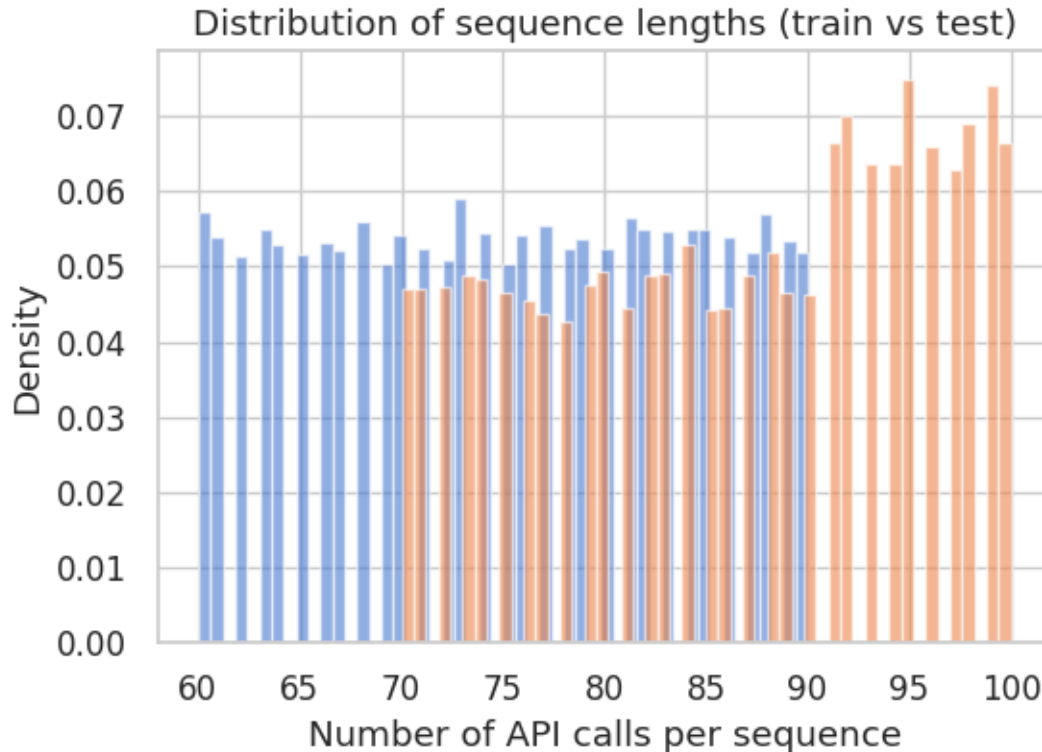
```

```
plt.title("Distribution of sequence lengths (train vs test)")

# Save the plot to the specified path
save_plot(plt.gcf(), f"distribution_sequence_lengths_train_test", save_dir)

plt.show()
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task2_plots/distribution_sequence_lengths_train_test.png



Q: Do you have the same number of API calls per sequence? If not, is the distribution of API calls per sequence the same for training and test sets? No, the sequences **do not** all have the same length in either the training or test sets.

- **Training set:** sequence lengths range from 60 to 90 API calls (mean = 75).
- **Test set:** sequence lengths range from 70 to 100 API calls (mean = 86).

The **sequences in the test set** are not only longer on average, but also fall within a range (90–100) that is completely absent from the training set. This discrepancy means that the model will have to generalize to sequence lengths it has never encountered during training, posing an additional challenge for sequential architectures.

Q: Can a FFNN handle a variable number of elements? If not, why? No, a feedforward neural network (FFNN) **cannot directly handle a variable number** of elements per input.

- FFNNs require a **fixed-size input vector**, because each input neuron corresponds to a specific feature, and the weight matrix has a fixed shape.
- If the number of elements (features) changes between samples, the network **cannot align inputs with the learned weights**, so it won't work properly.

Due to this architectural constraint, in order to process variable-length API sequences using a FFNN, they must be converted into fixed-length vectors using padding (to extend short sequences) or truncation (to shorten longer ones).

1.3.2 Fixed-Size Sequences

Q: How to estimate a fixed-size candidate? Which partition do you use to estimate it? To estimate a fixed-size input for an FFNN, it's possible to choose a representative sequence length from the training set only, since the model should not use information from the test set.

Common strategies include: 1. **Maximum length**: ensures all training sequences fit, but can be too large and waste memory. 2. **Median length**: gives a typical sequence size, but some longer sequences will need truncation. 3. **75th percentile length**: a compromise that covers most sequences without being overly large.

Among these strategies, we opted for the maximum training set length (90). This choice is justified by the fact that the distribution of lengths (as seen earlier) is fairly uniform and does not contain any extreme outliers; therefore, the computational “cost” of using 90 as a fixed size is offset by the advantage of not having to truncate (and thus lose) information in any training sequence.

```
[ ]: # Compute sequence lengths in training set
seq_lengths = df_train['api_call_sequence'].apply(len)

# Option 1: maximum length
max_len = seq_lengths.max()
print(f"Max length: {max_len}")

# Option 2: median
median_len = int(seq_lengths.median())
print(f"Median length: {median_len}")

# Option 3: 75th percentile
p75_len = int(np.percentile(seq_lengths, 75))
print(f"75th percentile length: {p75_len}")
```

Max length: 90

Median length: 75

75th percentile length: 83

Q: Given the estimate, what technique could you use to obtain the same number of API calls per sequence? To obtain the same number of API calls per sequence, the main strategies are:

1. **Padding:** Add “dummy” or special tokens to sequences **shorter than the fixed length** so they reach the target size.
2. **Truncation:** Cut sequences **longer than the fixed length** to fit the target size.

By combining padding and truncation based on the estimated fixed length, you can ensure that **all sequences have the same number of elements**, which is required for an FFNN.

```
[ ]: def pad_truncate_sequence(seq, fixed_len, pad_value=0):
    """
    Convert a variable-length sequence into a fixed-length sequence.

    Args:
        seq (list): original sequence (list of API calls)
        fixed_len (int): target length L
        pad_value: value to use for padding (0 is common for integers)

    Returns:
        list of length fixed_len
    """
    if len(seq) < fixed_len:
        # Pad at the end
        seq_padded = seq + [pad_value] * (fixed_len - len(seq))
    else:
        # Truncate at the end
        seq_padded = seq[:fixed_len]
    return seq_padded
```

```
[ ]: # --- Perform padding or truncating on the training data ---

L = max_len # chosen fixed size

# Apply the padding or truncating to all the dataframe
df_train['seq_fixed'] = df_train['api_call_sequence'].apply(lambda x: pad_truncate_sequence(x, L))
```

```
[ ]: # --- Inspect the train set (e.g. the first row) ---

print("First row of the train set before padding:")
print(df_train['api_call_sequence'].iloc[0])
print(f"Lenght of the sequence: {len(df_train['api_call_sequence'].iloc[0])}")
    ↪ # should be less than L

print("\nFirst row of the train set after padding:")
print(df_train['seq_fixed'].iloc[0])
print(f"Lenght of the sequence: {len(df_train['seq_fixed'].iloc[0])}") # should
    ↪ be exactly L
```

First row of the train set before padding:
 ['LdrGetDllHandle', 'LdrGetProcedureAddress', 'LdrLoadDll',

```

'LdrGetProcedureAddress', 'LdrGetDllHandle', 'LdrGetProcedureAddress',
'GetTimeZoneInformation', 'LoadStringW', 'RegOpenKeyExW', 'RegQueryValueExW',
'RegCloseKey', 'LdrGetDllHandle', 'LdrGetProcedureAddress', 'LdrGetDllHandle',
'LdrGetProcedureAddress', 'SetErrorMode', 'LdrLoadDll', 'SetErrorMode',
'LdrLoadDll', 'SetErrorMode', 'GetSystemMetrics', 'FindResourceExW',
'OleInitialize', 'FindResourceExW', 'LoadResource', 'FindResourceExW',
'LoadResource', 'NtClose', 'NtAllocateVirtualMemory', 'GetSystemMetrics',
'NtAllocateVirtualMemory', 'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrLoadDll',
'LdrGetProcedureAddress', 'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrLoadDll',
'LdrGetProcedureAddress', 'LdrLoadDll', 'LdrGetProcedureAddress',
'LookupAccountSidW', 'LdrGetProcedureAddress', 'LookupAccountSidW',
'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrLoadDll', 'LdrGetProcedureAddress',
'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrGetDllHandle', 'FindResourceExW',
'LoadResource', 'FindResourceExW', 'LoadResource', 'DrawTextExW',
'GetSystemMetrics', 'FindResourceExW', 'LoadResource', 'GetSystemMetrics',
'DrawTextExW', 'LdrGetDllHandle', 'FindResourceExW', 'LoadResource',
'FindResourceExW', 'LoadResource', 'FindResourceExW', 'LoadResource',
'LdrGetDllHandle', 'LdrGetProcedureAddress', 'LoadStringW', 'LdrGetDllHandle',
'LdrGetProcedureAddress']

```

Lenght of the sequence: 73

First row of the train set after padding:

```

['LdrGetDllHandle', 'LdrGetProcedureAddress', 'LdrLoadDll',
'LdrGetProcedureAddress', 'LdrGetDllHandle', 'LdrGetProcedureAddress',
'GetTimeZoneInformation', 'LoadStringW', 'RegOpenKeyExW', 'RegQueryValueExW',
'RegCloseKey', 'LdrGetDllHandle', 'LdrGetProcedureAddress', 'LdrGetDllHandle',
'LdrGetProcedureAddress', 'SetErrorMode', 'LdrLoadDll', 'SetErrorMode',
'LdrLoadDll', 'SetErrorMode', 'GetSystemMetrics', 'FindResourceExW',
'OleInitialize', 'FindResourceExW', 'LoadResource', 'FindResourceExW',
'LoadResource', 'NtClose', 'NtAllocateVirtualMemory', 'GetSystemMetrics',
'NtAllocateVirtualMemory', 'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrLoadDll',
'LdrGetProcedureAddress', 'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrLoadDll',
'LdrGetProcedureAddress', 'LdrLoadDll', 'LdrGetProcedureAddress',
'LookupAccountSidW', 'LdrGetProcedureAddress', 'LookupAccountSidW',
'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrLoadDll', 'LdrGetProcedureAddress',
'LdrLoadDll', 'LdrGetProcedureAddress', 'LdrGetDllHandle', 'FindResourceExW',
'LoadResource', 'FindResourceExW', 'LoadResource', 'DrawTextExW',
'GetSystemMetrics', 'FindResourceExW', 'LoadResource', 'GetSystemMetrics',
'DrawTextExW', 'LdrGetDllHandle', 'FindResourceExW', 'LoadResource',
'FindResourceExW', 'LoadResource', 'FindResourceExW', 'LoadResource',
'LdrGetDllHandle', 'LdrGetProcedureAddress', 'LoadStringW', 'LdrGetDllHandle',
'LdrGetProcedureAddress', 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

```

Lenght of the sequence: 90

Q: If at test time you have more API calls than the fixed-size, what do you do with the exceeding API calls? At test time, if a sequence has **more API calls than the fixed size**, the exceeding API calls should be **truncated** to fit the fixed length. Sequences shorter

than the fixed size are **padded** with dummy tokens.

We have decided to perform the same pad or truncate action on the test set, so that all sequences have a consistent length compatible with the FFNN.

```
[ ]: # To ensure consistency, we apply the padding or truncating to also on the test
      ↪df
df_test['seq_fixed'] = df_test['api_call_sequence'].apply(lambda x:
      ↪pad_truncate_sequence(x, L))
```

```
[ ]: df_train
```

```
[ ]:
      api_call_sequence  is_malware  \
0      [LdrGetDllHandle, LdrGetProcedureAddress, LdrL...      1
1      [NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...      1
2      [FindResourceExW, LoadResource, FindResourceEx...      1
3      [FindResourceExW, LoadResource, FindResourceEx...      1
4      [LdrGetProcedureAddress, SetErrorMode, LdrLoad...      1
...      ...      ...
16320  [LdrGetProcedureAddress, LdrLoadDll, LdrGetPro...      1
16321  [NtClose, LdrGetProcedureAddress, CryptCreateH...      1
16322  [LdrGetProcedureAddress, LdrGetDllHandle, LdrG...      1
16323  [LdrGetProcedureAddress, LdrGetDllHandle, LdrG...      1
16324  [SetFilePointer, NtReadFile, SetFilePointer, N...      1

      seq_length      seq_fixed
0              73  [LdrGetDllHandle, LdrGetProcedureAddress, LdrL...
1              88  [NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...
2              79  [FindResourceExW, LoadResource, FindResourceEx...
3              71  [FindResourceExW, LoadResource, FindResourceEx...
4              63  [LdrGetProcedureAddress, SetErrorMode, LdrLoad...
...      ...      ...
16320      64  [LdrGetProcedureAddress, LdrLoadDll, LdrGetPro...
16321      78  [NtClose, LdrGetProcedureAddress, CryptCreateH...
16322      84  [LdrGetProcedureAddress, LdrGetDllHandle, LdrG...
16323      62  [LdrGetProcedureAddress, LdrGetDllHandle, LdrG...
16324      77  [SetFilePointer, NtReadFile, SetFilePointer, N...

[16325 rows x 4 columns]
```

```
[ ]: df_test
```

```
[ ]:
      api_call_sequence  is_malware  \
0      [NtQueryValueKey, NtClose, NtOpenKey, NtQueryV...      1
1      [LdrGetProcedureAddress, NtClose, NtOpenKey, N...      1
2      [NtOpenKey, NtQueryValueKey, NtClose, NtOpenKe...      1
3      [NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...      1
4      [NtOpenKey, NtQueryValueKey, NtClose, LdrGetPr...      1
```

```

...
6500 [SetErrorMode, NtOpenFile, NtClose, SHGetFolde... 0
6501 [NtProtectVirtualMemory, RegOpenKeyExW, RegQue... 1
6502 [RegOpenKeyExW, RegQueryValueExW, RegCloseKey,... 1
6503 [NtQueryValueKey, NtClose, RegOpenKeyExA, RegQ... 1
6504 [NtClose, LdrGetProcedureAddress, CryptCreateH... 1

      seq_length      seq_fixed
0          98 [NtQueryValueKey, NtClose, NtOpenKey, NtQueryV...
1          98 [LdrGetProcedureAddress, NtClose, NtOpenKey, N...
2          99 [NtOpenKey, NtQueryValueKey, NtClose, NtOpenKe...
3          85 [NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...
4          80 [NtOpenKey, NtQueryValueKey, NtClose, LdrGetPr...
...
6500      75 [SetErrorMode, NtOpenFile, NtClose, SHGetFolde...
6501      83 [NtProtectVirtualMemory, RegOpenKeyExW, RegQue...
6502      70 [RegOpenKeyExW, RegQueryValueExW, RegCloseKey,...
6503      96 [NtQueryValueKey, NtClose, RegOpenKeyExA, RegQ...
6504      78 [NtClose, LdrGetProcedureAddress, CryptCreateH...

```

[6505 rows x 4 columns]

1.3.3 Handling Categorical Features

```

[ ]: # --- Build a unified vocabulary ---

all_sequences = df_train['seq_fixed']

# Flatten all sequences into a single list of API calls
all_api_calls = [api for seq in all_sequences for api in seq]

[ ]: # --- Build vocabulary with padding handling ---

# 1. Filter strings only. We ignore 0 here because we will manually assign it
    ↪ later.
unique_api_calls = sorted(list(set(api for api in all_api_calls if
    ↪ isinstance(api, str))))

# 2. Define Special Tokens
PAD_IDX = 0 # <PAD>
UNK_IDX = 1 # <UNK>

# 3. Map API call -> sequential ID (Start from 2 to save space for PAD and UNK)
api_to_id = {api: i + 2 for i, api in enumerate(unique_api_calls)}

# 4. Add the special tokens to the dictionary
# (Optional: helpful if you need to reverse look up '<PAD>' later)

```



```

api_to_id['<PAD>'] = PAD_IDX
api_to_id['<UNK>'] = UNK_IDX

# Map sequential ID -> API call (for verification/debugging)
id_to_api = {i: api for api, i in api_to_id.items()}

print(f"Number of unique API calls (including special tokens): {len(api_to_id)}")
print("\nExample mapping (API -> ID):", list(api_to_id.items())[:5])
print("\nExample mapping (ID -> API):", list(id_to_api.items())[:5])

```

Number of unique API calls (including special tokens): 260

Example mapping (API -> ID): [('CertOpenStore', 2), ('CertOpenSystemStoreW', 3), ('CoCreateInstance', 4), ('CoCreateInstanceEx', 5), ('CoGetClassObject', 6)]

Example mapping (ID -> API): [(2, 'CertOpenStore'), (3, 'CertOpenSystemStoreW'), (4, 'CoCreateInstance'), (5, 'CoCreateInstanceEx'), (6, 'CoGetClassObject')]

Sequential Identifiers Approach

```

[ ]: def sequence_to_ids(seq, mapping, pad_idx, unk_idx):
    """
    Convert a sequence of API calls to their corresponding IDs.
    - If item is 0 (integer padding), map to pad_idx.
    - If item is string but not in mapping, map to unk_idx.
    """
    ids = []
    for item in seq:
        # Case A: It is existing padding (integer 0)
        if item == 0:
            ids.append(pad_idx)
        # Case B: It is a known string API
        elif item in mapping:
            ids.append(mapping[item])
        # Case C: It is an unknown string API (Test set OOV)
        else:
            ids.append(unk_idx)
    return ids

[ ]: # --- Convert sequences to sequential IDs ---

# Apply conversion to train and test sets
# Note: We pass UNK_IDX (1) so unknown calls in Test set become 1.
df_train['seq_sequential_ids'] = df_train['seq_fixed'].apply(
    lambda seq: sequence_to_ids(seq, api_to_id, PAD_IDX, UNK_IDX)
)

```

```

df_test['seq_sequential_ids'] = df_test['seq_fixed'].apply(
    lambda seq: sequence_to_ids(seq, api_to_id, PAD_IDX, UNK_IDX)
)

# Quick check
print("\nSequential IDs - first row of train set:")
print(df_train['seq_sequential_ids'].iloc[0])
print("Length of the sequence:", len(df_train['seq_sequential_ids'].iloc[0]))

print("\nSequential IDs - first row of test set:")
print(df_test['seq_sequential_ids'].iloc[0])
print("Length of the sequence:", len(df_test['seq_sequential_ids'].iloc[0]))

# Verify that the logic works (check for any None values)
# If this prints nothing, your data is clean!
for seq in df_test['seq_sequential_ids']:
    if None in seq:
        print("Error: Found None in sequence! Check mapping.")
        break

```

Sequential IDs - first row of train set:

```

[117, 118, 119, 118, 117, 118, 86, 123, 201, 205, 188, 117, 118, 117, 118, 219,
119, 219, 119, 219, 81, 46, 179, 46, 121, 46, 121, 134, 133, 81, 133, 119, 118,
119, 118, 119, 118, 119, 118, 119, 118, 124, 118, 124, 119, 118, 119, 118, 119,
118, 117, 46, 121, 46, 121, 39, 81, 46, 121, 81, 39, 117, 46, 121, 46, 121, 46,
121, 117, 118, 123, 117, 118, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

```

Length of the sequence: 90

Sequential IDs - first row of test set:

```

[164, 134, 152, 164, 134, 159, 122, 133, 122, 117, 118, 117, 118, 117, 122, 118,
81, 46, 121, 46, 121, 46, 121, 46, 121, 46, 121, 46, 121, 46, 121, 46, 121, 46,
121, 46, 121, 46, 121, 46, 121, 46, 121, 46, 121, 118, 46, 121, 46, 121, 133,
81, 134, 81, 133, 119, 118, 117, 46, 121, 46, 121, 39, 81, 46, 121, 81, 39, 117,
46, 121, 46, 121, 46, 121, 117, 118, 122, 117, 118, 117, 118, 81, 44, 121, 229,
133, 119, 118, 84]

```

Length of the sequence: 90

```
[ ]: df_train
```

```

[ ]:
      api_call_sequence  is_malware  \
0  [LdrGetDllHandle, LdrGetProcedureAddress, LdrL...      1
1  [NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...      1
2  [FindResourceExW, LoadResource, FindResourceEx...      1
3  [FindResourceExW, LoadResource, FindResourceEx...      1
4  [LdrGetProcedureAddress, SetErrorMode, LdrLoad...      1
...      ...      ...
16320 [LdrGetProcedureAddress, LdrLoadDll, LdrGetPro...      1

```

16321	[NtClose, LdrGetProcedureAddress, CryptCreateH...	1
16322	[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...	1
16323	[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...	1
16324	[SetFilePointer, NtReadFile, SetFilePointer, N...	1

	seq_length		seq_fixed \
0	73	[LdrGetDllHandle, LdrGetProcedureAddress, LdrL...	
1	88	[NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...	
2	79	[FindResourceExW, LoadResource, FindResourceEx...	
3	71	[FindResourceExW, LoadResource, FindResourceEx...	
4	63	[LdrGetProcedureAddress, SetErrorMode, LdrLoad...	
...	...	...	
16320	64	[LdrGetProcedureAddress, LdrLoadDll, LdrGetPro...	
16321	78	[NtClose, LdrGetProcedureAddress, CryptCreateH...	
16322	84	[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...	
16323	62	[LdrGetProcedureAddress, LdrGetDllHandle, LdrG...	
16324	77	[SetFilePointer, NtReadFile, SetFilePointer, N...	

	seq_sequential_ids
0	[117, 118, 119, 118, 117, 118, 86, 123, 201, 2...
1	[133, 119, 118, 82, 80, 137, 82, 152, 153, 163...
2	[46, 121, 46, 121, 46, 121, 46, 121, 46, 121, ...
3	[46, 121, 46, 121, 46, 121, 46, 121, 46, 121, ...
4	[118, 219, 119, 219, 81, 46, 121, 46, 121, 46,...
...	...
16320	[118, 119, 118, 119, 118, 119, 118, 119, 118, ...
16321	[134, 118, 24, 118, 30, 118, 135, 66, 221, 166...
16322	[118, 117, 118, 81, 216, 119, 118, 46, 121, 46...
16323	[118, 117, 118, 117, 118, 117, 118, 117, 118, ...
16324	[221, 166, 221, 166, 221, 45, 121, 133, 146, 1...

[16325 rows x 5 columns]

```
[ ]: df_test
```

	api_call_sequence	is_malware \
0	[NtQueryValueKey, NtClose, NtOpenKey, NtQueryV...	1
1	[LdrGetProcedureAddress, NtClose, NtOpenKey, N...	1
2	[NtOpenKey, NtQueryValueKey, NtClose, NtOpenKe...	1
3	[NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...	1
4	[NtOpenKey, NtQueryValueKey, NtClose, LdrGetPr...	1
...	...	...
6500	[SetErrorMode, NtOpenFile, NtClose, SHGetFolde...	0
6501	[NtProtectVirtualMemory, RegOpenKeyExW, RegQue...	1
6502	[RegOpenKeyExW, RegQueryValueExW, RegCloseKey,...	1
6503	[NtQueryValueKey, NtClose, RegOpenKeyExA, RegQ...	1
6504	[NtClose, LdrGetProcedureAddress, CryptCreateH...	1

```

seq_length                                seq_fixed \
0      98 [NtQueryValueKey, NtClose, NtOpenKey, NtQueryV...
1      98 [LdrGetProcedureAddress, NtClose, NtOpenKey, N...
2      99 [NtOpenKey, NtQueryValueKey, NtClose, NtOpenKe...
3      85 [NtAllocateVirtualMemory, LdrLoadDll, LdrGetPr...
4      80 [NtOpenKey, NtQueryValueKey, NtClose, LdrGetPr...
...      ...
6500    75 [SetErrorMode, NtOpenFile, NtClose, SHGetFolde...
6501    83 [NtProtectVirtualMemory, RegOpenKeyExW, RegQue...
6502    70 [RegOpenKeyExW, RegQueryValueExW, RegCloseKey,...
6503    96 [NtQueryValueKey, NtClose, RegOpenKeyExA, RegQ...
6504    78 [NtClose, LdrGetProcedureAddress, CryptCreateH...

                                seq_sequential_ids
0      [164, 134, 152, 164, 134, 159, 122, 133, 122, ...
1      [118, 134, 152, 164, 134, 152, 164, 134, 119, ...
2      [152, 164, 134, 152, 164, 134, 117, 118, 117, ...
3      [133, 119, 118, 119, 118, 82, 200, 204, 188, 1...
4      [152, 164, 134, 118, 24, 118, 30, 118, 135, 66...
...      ...
6500    [219, 151, 134, 214, 219, 63, 219, 151, 134, 1...
6501    [158, 201, 205, 188, 140, 133, 158, 133, 146, ...
6502    [201, 205, 188, 201, 205, 188, 80, 117, 118, 2...
6503    [164, 134, 200, 204, 188, 153, 164, 134, 119, ...
6504    [134, 118, 24, 118, 30, 118, 135, 66, 221, 166...

[6505 rows x 5 columns]

```

Learnable Embeddings Approach

```

[ ]: # The learnable embeddings approach is implemented in the first layer of the
      ↪neural network model.
      # The nn.Embedding layer in the MalwareFFNN class handles this process.

```

1.3.4 FFNN Training

```

[ ]: class MalwareFFNN(nn.Module):
      def __init__(self, seq_len, hidden_dims=[128,64,32],
                    dropout_rates=None, batchnorm_flags=None,
                    use_embedding=True, vocab_size=None, embedding_dim=16,
                    dropout_embedding_rate=0.1):
          """
          Flexible FFNN for malware classification.

          Parameters:
          - seq_len: length of each API call sequence

```

```

        - vocab_size: number of unique API calls (required if
↪use_embedding=True)
        - hidden_dims: list of hidden layer sizes
        - embedding_dim: size of embedding vectors (only used if
↪use_embedding=True)
        - use_embedding: whether to use an embedding layer (True) or raw
↪sequential IDs / one-hot (False)
    """
    super(MalwareFFNN, self).__init__()
    self.use_embedding = use_embedding

    if self.use_embedding:
        if vocab_size is None:
            raise ValueError("vocab_size must be provided if
↪use_embedding=True")

        # Embedding layer for sequential IDs
        self.embedding = nn.Embedding(num_embeddings=vocab_size,
↪embedding_dim=embedding_dim, padding_idx=0)
        input_dim = seq_len * embedding_dim

        self.dropout_embed = nn.Dropout(dropout_embedding_rate)
    else:
        # Input will be one-hot encoded or numeric vector
        input_dim = seq_len # assumes IDs are already numeric

    # Ensure defaults if none are provided
    if dropout_rates is None:
        dropout_rates = [0.0] * len(hidden_dims) # no dropout by default
    if batchnorm_flags is None:
        batchnorm_flags = [False] * len(hidden_dims) # no batchnorm by
↪default

    # Safety check for lengths
    assert len(dropout_rates) == len(hidden_dims), \
        "dropout_rates length must match hidden_dims length"
    assert len(batchnorm_flags) == len(hidden_dims), \
        "batchnorm_flags length must match hidden_dims length"

    # Build hidden layers
    layers = []
    prev_dim = input_dim
    for h_dim, dropout_rate, use_bn in zip(hidden_dims, dropout_rates,
↪batchnorm_flags):
        layers.append(nn.Linear(prev_dim, h_dim))
        if use_bn:

```

```

        layers.append(nn.BatchNorm1d(h_dim))
    layers.append(nn.ReLU())
    if dropout_rate > 0:
        layers.append(nn.Dropout(dropout_rate))
    prev_dim = h_dim
    self.ffnn = nn.Sequential(*layers)

    # Output layer: binary classification
    self.output_layer = nn.Linear(prev_dim, 1)

def forward(self, x):
    if self.use_embedding:
        # x: [batch_size, seq_len] of IDs
        x = self.embedding(x)      # [batch_size, seq_len, embedding_dim]
        x = self.dropout_embed(x)
        x = x.view(x.size(0), -1) # flatten: [batch_size, ↵
↵seq_len*embedding_dim]
    else:
        # x: [batch_size, seq_len] or [batch_size, seq_len*vocab_size] if ↵
↵one-hot flattened
        x = x.float()

    x = self.ffnn(x)
    out = self.output_layer(x) # [batch_size, 1]
    return out

```

```

[ ]: # --- Training function with early stopping ---

def train_model(model, train_loader, val_loader, epochs, optimizer, criterion, ↵
↵min_delta=None, patience=None, scheduler=None):
    """
    Train a PyTorch model with early stopping and metric tracking.
    Returns: model, history dictionary (losses + metrics), and elapsed time.
    """

    # --- DEVICE DETECTION ---
    device = next(model.parameters()).device

    # --- HISTORY STORAGE ---
    history = {
        'train_loss': [],
        'val_loss': [],
        'val_f1_macro': [],
        'val_f1_weighted': [],
    }

    best_val_loss = float('inf')

```

```

best_model_state = None
trigger_times = 0

start_time = time.time()

for epoch in range(epochs):
    # --- TRAINING ---
    model.train()
    batch_losses = []
    for X_batch, y_batch in train_loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)
        optimizer.zero_grad()
        outputs = model(X_batch)
        loss = criterion(outputs, y_batch)
        loss.backward()
        optimizer.step()
        batch_losses.append(loss.item())
    train_loss = np.mean(batch_losses) if batch_losses else 0.0

    # --- VALIDATION & METRICS ---
    model.eval()
    val_batch_losses = []
    all_preds = []
    all_targets = []

    with torch.no_grad():
        for X_batch, y_batch in val_loader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            outputs = model(X_batch)
            loss = criterion(outputs, y_batch)
            val_batch_losses.append(loss.item())

        # Convert logits to probabilities and then binary predictions
        ↪ (0 or 1)

        # Assumes Binary Classification (BCEWithLogitsLoss)
        probs = torch.sigmoid(outputs)
        preds = (probs > 0.5).float()

        # Store for epoch-level metric calculation
        all_preds.extend(preds.cpu().numpy())
        all_targets.extend(y_batch.cpu().numpy())

    val_loss = np.mean(val_batch_losses) if val_batch_losses else 0.0

    # Calculate Metrics for this Epoch
    epoch_f1_macro = f1_score(all_targets, all_preds, average='macro', ↪
    ↪ zero_division=0)

```

```

        epoch_f1_weighted = f1_score(all_targets, all_preds,
↪average='weighted', zero_division=0)

        # Update History
        history['train_loss'].append(train_loss)
        history['val_loss'].append(val_loss)
        history['val_f1_macro'].append(epoch_f1_macro)
        history['val_f1_weighted'].append(epoch_f1_weighted)

        # --- SCHEDULER STEP ---
        if scheduler is not None:
            last_lr = optimizer.param_groups[0]['lr']
            if isinstance(scheduler, torch.optim.lr_scheduler.
↪ReduceLROnPlateau):
                scheduler.step(val_loss)
            else:
                scheduler.step()
            curr_lr = optimizer.param_groups[0]['lr']
            if curr_lr != last_lr:
                print(f"Epoch {epoch}: LR reduced from {last_lr:.6f} to
↪{curr_lr:.6f}")

        # --- EARLY STOPPING ---
        if min_delta is not None:
            if val_loss < best_val_loss - min_delta:
                best_val_loss = val_loss
                # Save best model state
                best_model_state = {k: v.cpu().clone() for k, v in model.
↪state_dict().items()}
                trigger_times = 0
            else:
                trigger_times += 1
                if trigger_times >= patience:
                    print(f"Early stopping at epoch {epoch+1} (best val loss:
↪{best_val_loss:.6f})")
                    break

        if (epoch+1) % 5 == 0 or epoch == 0 or epoch == epochs:
            print(f"Epoch {epoch+1}/{epochs} - Train Loss: {train_loss:.4f},
↪Val Loss: {val_loss:.4f}, Val F1(W): {epoch_f1_weighted:.4f}")

        # Restore best model after training loop completes
        if best_model_state is not None:
            model.load_state_dict(best_model_state)
            model.to(device)

```



```

    elapsed = time.time() - start_time
    elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed))
    print(f'\nTraining completed in {elapsed_str} (hh:mm:ss) - {elapsed:.2f}␣
    ↪seconds')

    return model, history, elapsed

```

```

[ ]: def evaluate_model(model, X_tensor, y_true, model_name: str = "Unnamed model"):
    """
    Evaluate a trained model on a given dataset and return the classification␣
    ↪report.

    Handles missing predicted classes gracefully (zero_division=0) and
    reports which classes were not predicted, along with the model/config name.
    """
    model.eval()
    with torch.no_grad():
        outputs = model(X_tensor)
        probs = torch.sigmoid(outputs)
        y_pred = (probs > 0.5).long().cpu().numpy().flatten() # Binary␣
    ↪prediction (0 or 1)

    # Convert y_true to numpy if it's a tensor
    if isinstance(y_true, torch.Tensor):
        y_true = y_true.cpu().numpy().flatten()

    # Identify missing classes (not predicted at all)
    missing_classes = set(np.unique(y_true)) - set(np.unique(y_pred))
    if missing_classes:
        missing_classes = [int(x) for x in sorted(missing_classes)]
        print(f"Warning: {model_name} made no predictions for classes:␣
    ↪{missing_classes}")

    # Generate classification report without raising warnings
    report = classification_report(y_true, y_pred, digits=4, zero_division=0)

    return report

```

```

[ ]: def plot_cm_task2(model, X_tensor, y_tensor, model_name, save_dir=None):
    """
    Specific for FFNN. Takes tensors, moves to device, predicts, and plots.
    """
    # Ensure model is in eval mode
    model.eval()

    # Detect device (cuda or cpu) from the model parameters
    device = next(model.parameters()).device

```

```

# Move inputs to the same device as model
X_input = X_tensor.to(device)

with torch.no_grad():
    # Get logits
    outputs = model(X_input)
    # Apply sigmoid to get probabilities
    probs = torch.sigmoid(outputs)
    # Convert to binary predictions (0 or 1)
    y_pred = (probs > 0.5).long().cpu().numpy().flatten()

# Ensure y_true is on CPU and numpy
y_true = y_tensor.cpu().numpy().flatten()

# Plot
_internal_plot_cm(y_true, y_pred, model_name, save_dir)

```

```

[ ]: def plot_training_metrics(histories, model_names=None, save_dir=None):
    """
    Compare multiple models' validation F1 Macro across epochs.
    Only shows validation F1 macro (no weighted, no markers for cleaner
    ↪ plots).
    """
    # Normalize input
    if isinstance(histories, dict):
        hist_map = histories
        names = [f"lr_{k:.0e}" if isinstance(k, float) else str(k) for k in
        ↪ hist_map.keys()]
        hist_map = dict(zip(names, hist_map.values()))
    else:
        if model_names is None:
            names = [f"Model_{i+1}" for i in range(len(histories))]
        else:
            names = list(model_names)
            hist_map = dict(zip(names, histories))

    # Validate
    for name, h in hist_map.items():
        if 'val_f1_macro' not in h:
            raise KeyError(f"History for '{name}' must contain 'val_f1_macro'.
            ↪")

    colors = ['#E74C3C', '#F39C12', '#27AE60', '#3498DB', '#9B59B6',
    ↪ '#34495E']

    fig, ax = plt.subplots(figsize=(10, 5))

```

```

    for idx, (name, h) in enumerate(hist_map.items()):
        epochs = range(1, len(h['val_f1_macro']) + 1)
        color = colors[idx % len(colors)]
        ax.plot(epochs, h['val_f1_macro'], label=name, linewidth=2,
↪color=color)

    ax.set_title("Validation F1 Macro (LR Comparison)", fontsize=13)
    ax.set_xlabel("Epochs", fontsize=11)
    ax.set_ylabel("F1 Macro", fontsize=11)
    ax.grid(True, alpha=0.3)
    ax.legend(fontsize=9)
    plt.tight_layout()

    if save_dir:
        save_plot(fig, filename=plot_name, path=save_dir, fmt="png",
↪dpi=300, close_fig=False)
    plt.show()

```

```

[ ]: # --- Plotting function for LR comparison ---
def plot_lr_comparison(histories_dict, model_name, save_dir=None):
    """
    Plot ONLY validation losses for different learning rates.
    (When multiple LR's are present, we show only val curves for readability.)
    """
    plt.style.use('seaborn-v0_8-paper')

    fig, ax = plt.subplots(figsize=(8, 5))

    colors = ['#E74C3C', '#F39C12', '#27AE60', '#3498DB', '#9B59B6',
↪'#34495E']

    for idx, (lr, history) in enumerate(sorted(histories_dict.items(),
↪reverse=True)):
        color = colors[idx % len(colors)]
        label = f'lr_{lr:.0e}'
        ax.plot(history['val_loss'], color=color, linewidth=2, label=label,
↪alpha=0.8)

    ax.set_title(f'{model_name} - Validation Loss (LR Comparison)',
↪fontsize=13, pad=10)
    ax.set_ylabel('Validation Loss', fontsize=10)
    ax.set_xlabel('Epochs', fontsize=10)
    ax.grid(True, linestyle='--', alpha=0.3)
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
    ax.legend(fontsize=9, loc='best')

```

```

plt.tight_layout()

if save_dir:
    save_plot(fig, f'{model_name.lower().replace(" ", "_")}_lr_comparison', save_dir)

plt.show()

```

```

[ ]: # --- Plot Train vs Val ---

def plot_train_val_grid(histories_dict, model_name, save_dir=None):
    """
    Plots a grid (1 row, N columns) showing Train vs Val loss for EACH learning
    ↪rate.
    This helps diagnose overfitting/underfitting for each specific setting.
    Works for both FFNN (Task 2) and RNN (Task 3) models.

    Args:
        histories_dict: Dictionary with LR values as keys and history dicts as
        ↪values
        model_name: Name of the model architecture
        save_dir: Directory to save the plot
    """
    lrs = sorted(histories_dict.keys(), reverse=True) # Highest LR first
    num_lrs = len(lrs)

    # Set style
    plt.style.use('seaborn-v0_8-paper')

    # Create figure with 1 row and N columns
    fig, axes = plt.subplots(1, num_lrs, figsize=(5 * num_lrs, 4), sharey=True)

    # Handle case if there is only 1 LR (axes is not a list)
    if num_lrs == 1:
        axes = [axes]

    # Common colors
    col_train = '#2980B9' # Blue
    col_val = '#E74C3C' # Red

    for idx, lr in enumerate(lrs):
        ax = axes[idx]
        history = histories_dict[lr]
        epochs = range(1, len(history['train_loss']) + 1)

        # Plot Lines

```

```

        ax.plot(epochs, history['train_loss'], label='Train Loss',
        ↪color=col_train, linewidth=2)
        ax.plot(epochs, history['val_loss'], label='Val Loss', color=col_val,
        ↪linewidth=2)

        # Find best epoch (min val loss)
        min_val_loss = min(history['val_loss'])
        best_epoch = history['val_loss'].index(min_val_loss) + 1

        # Mark best epoch
        ax.axvline(x=best_epoch, color='green', linestyle=':', alpha=0.6,
        ↪label=f'Best Ep: {best_epoch}')
        ax.scatter(best_epoch, min_val_loss, color='green', zorder=5)

        # Styling
        ax.set_title(f'LR: {lr:.0e}', fontsize=12, fontweight='bold')
        ax.set_xlabel('Epochs')
        if idx == 0:
            ax.set_ylabel('Loss')

        ax.legend(fontsize=9)
        ax.grid(True, linestyle='-', alpha=0.3)
        ax.spines['top'].set_visible(False)
        ax.spines['right'].set_visible(False)

        fig.suptitle(f'{model_name} - Overfitting Diagnosis (Train vs Val)',
        ↪fontsize=14, y=1.05)
        plt.tight_layout()

        if save_dir:
            save_plot(fig, f'{model_name.lower().replace(" ", "_")}_diagnosis',
            ↪save_dir)

        plt.show()

```

Training with Sequential Identifiers Approach

```

[ ]: # --- Create a validation set ---

# Convert sequences to a numpy array (shape: [samples, seq_len])
X_train = np.array(df_train['seq_sequential_ids'].tolist())
X_test = np.array(df_test['seq_sequential_ids'].tolist())

# Labels
y_train = df_train['is_malware'].values
y_test = df_test['is_malware'].values

```

```

# Split train into train + val
X_train_split, X_val, y_train_split, y_val = train_test_split(
    X_train, y_train, test_size=0.2, random_state=42, stratify=y_train
)

print("Train set:", X_train_split.shape, y_train_split.shape)
print("Validation set:", X_val.shape, y_val.shape)

```

```

Train set: (13060, 90) (13060,)
Validation set: (3265, 90) (3265,)

```

```

[ ]: # --- Prepare PyTorch datasets and loaders ---

# Convert to tensors
X_train_tensor = torch.tensor(X_train_split.astype(np.int64), dtype=torch.long)
y_train_tensor = torch.tensor(y_train_split, dtype=torch.float32).unsqueeze(1)

X_val_tensor = torch.tensor(X_val.astype(np.int64), dtype=torch.long)
y_val_tensor = torch.tensor(y_val, dtype=torch.float32).unsqueeze(1)

X_test_tensor = torch.tensor(X_test.astype(np.int64), dtype=torch.long)
y_test_tensor = torch.tensor(y_test, dtype=torch.float32).unsqueeze(1)

# Create datasets and loaders
batch_size = 64

train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
val_dataset = TensorDataset(X_val_tensor, y_val_tensor)
test_dataset = TensorDataset(X_test_tensor, y_test_tensor)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=True)

```

```

[ ]: unique, counts = np.unique(y_train_split, return_counts=True)
print(dict(zip(unique, counts)))

```

```
{np.int64(0): np.int64(489), np.int64(1): np.int64(12571)}
```

```

[ ]: def compute_pos_weight(y):
    """
    Compute positive class weight for BCEWithLogitsLoss.
    Returns a tensor > 1 if positive class is minority.
    """
    y_np = np.array(y, dtype=int)
    class_counts = np.bincount(y_np)

    if len(class_counts) < 2:

```

```

        raise ValueError("Dataset must contain both classes 0 and 1.")

    count_neg, count_pos = class_counts[0], class_counts[1]

    # if 1 = malware (majority class)
    ratio = count_neg / count_pos

    # Optionally stabilize the weight
    ratio = max(ratio, 0.2)

    print(f"Negatives: {count_neg}, Positives: {count_pos}")
    print(f"pos_weight (applied to y=1 samples): {ratio:.4f}")

    # return torch.tensor([ratio], dtype=torch.float32)
    return torch.tensor([ratio], dtype=torch.float32)

```

```

[ ]: # --- Configuration Constants ---
seq_len = X_train.shape[1]
hidden_dims = [256, 128, 64]
dropout_rates = [0.4, 0.35, 0.3]
batchnorm_flags = [True, True, True]
min_delta = 0.00001
patience_early_stop = 15
epochs = 100

# --- Device Setup ---
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")

# Compute weights and move to device
pos_weight = compute_pos_weight(y_train_split).to(device)

# --- Define LR Candidates ---
learning_rates_seq = [0.0001, 0.00005, 0.00001]
histories_seq = {} # Store results here
models_seq = {} # Store models here
elapsed_seq_times = {} # Store elapsed times here

print(f"\n{'='*80}")
print("Training FFNN (Sequential IDs) with different learning rates")
print(f"{'='*80}")

for lr in learning_rates_seq:
    print(f"\n--- Training FFNN (Sequential IDs) with lr={lr:.0e} ---")

    # 1. Re-Initialize Model and move to device
    model_seq = MalwareFFNN(

```

```

        seq_len,
        hidden_dims,
        dropout_rates,
        batchnorm_flags,
        use_embedding=False,
        dropout_embedding_rate=0.1
    ).to(device)

# 2. Re-Initialize Criterion
criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)

# 3. Initialize Optimizer with current LR
optimizer = optim.AdamW(model_seq.parameters(), lr=lr, weight_decay=1e-5)

# 4. Initialize Scheduler
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode='min',
    factor=0.7,
    patience=10,
    min_lr=1e-6
)

# 5. Train
model_seq, history, elapsed_seq = train_model(
    model_seq,
    train_loader,
    val_loader,
    epochs,
    optimizer,
    criterion,
    min_delta,
    patience=patience_early_stop,
    scheduler=scheduler
)

# Store model and history with numeric LR key
models_seq[lr] = model_seq
histories_seq[lr] = history
elapsed_seq_times[lr] = elapsed_seq

elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed_seq))
print(f"Completed. Training time: {elapsed_str} ({elapsed_seq:.2f}s) -_
↳Final val loss: {history['val_loss'][-1]:.4f}")

```

Using device: cuda

Negatives: 489, Positives: 12571

pos_weight (applied to y=1 samples): 0.2000

=====

Training FFNN (Sequential IDs) with different learning rates

=====

--- Training FFNN (Sequential IDs) with lr=1e-04 ---

Epoch 1/100 - Train Loss: 0.1485, Val Loss: 0.1425, Val F1(W): 0.8917
Epoch 5/100 - Train Loss: 0.1007, Val Loss: 0.1052, Val F1(W): 0.9448
Epoch 10/100 - Train Loss: 0.0916, Val Loss: 0.0977, Val F1(W): 0.9366
Epoch 15/100 - Train Loss: 0.0894, Val Loss: 0.0953, Val F1(W): 0.9309
Epoch 20/100 - Train Loss: 0.0841, Val Loss: 0.0946, Val F1(W): 0.9291
Epoch 25/100 - Train Loss: 0.0832, Val Loss: 0.0932, Val F1(W): 0.9248
Epoch 30/100 - Train Loss: 0.0779, Val Loss: 0.0961, Val F1(W): 0.9212
Epoch 35/100 - Train Loss: 0.0744, Val Loss: 0.0924, Val F1(W): 0.9237
Epoch 40/100 - Train Loss: 0.0733, Val Loss: 0.0935, Val F1(W): 0.9204
Epoch 41: LR reduced from 0.000100 to 0.000070
Epoch 45/100 - Train Loss: 0.0705, Val Loss: 0.0953, Val F1(W): 0.9262
Early stopping at epoch 46 (best val loss: 0.091867)

Training completed in 00:00:39 (hh:mm:ss) - 39.06 seconds

Completed. Training time: 00:00:39 (39.06s) - Final val loss: 0.0945

--- Training FFNN (Sequential IDs) with lr=5e-05 ---

Epoch 1/100 - Train Loss: 0.2339, Val Loss: 0.1794, Val F1(W): 0.1076
Epoch 5/100 - Train Loss: 0.1423, Val Loss: 0.1414, Val F1(W): 0.8938
Epoch 10/100 - Train Loss: 0.1061, Val Loss: 0.1143, Val F1(W): 0.9420
Epoch 15/100 - Train Loss: 0.0957, Val Loss: 0.1030, Val F1(W): 0.9407
Epoch 20/100 - Train Loss: 0.0916, Val Loss: 0.0977, Val F1(W): 0.9401
Epoch 25/100 - Train Loss: 0.0890, Val Loss: 0.0957, Val F1(W): 0.9354
Epoch 30/100 - Train Loss: 0.0876, Val Loss: 0.0953, Val F1(W): 0.9339
Epoch 35/100 - Train Loss: 0.0859, Val Loss: 0.0947, Val F1(W): 0.9294
Epoch 40/100 - Train Loss: 0.0835, Val Loss: 0.0951, Val F1(W): 0.9227
Epoch 45/100 - Train Loss: 0.0810, Val Loss: 0.0929, Val F1(W): 0.9236
Epoch 48: LR reduced from 0.000050 to 0.000035
Epoch 50/100 - Train Loss: 0.0801, Val Loss: 0.0930, Val F1(W): 0.9236
Early stopping at epoch 53 (best val loss: 0.092690)

Training completed in 00:00:44 (hh:mm:ss) - 44.30 seconds

Completed. Training time: 00:00:44 (44.30s) - Final val loss: 0.0950

--- Training FFNN (Sequential IDs) with lr=1e-05 ---

Epoch 1/100 - Train Loss: 0.1586, Val Loss: 0.1561, Val F1(W): 0.7484
Epoch 5/100 - Train Loss: 0.1431, Val Loss: 0.1455, Val F1(W): 0.8914
Epoch 10/100 - Train Loss: 0.1293, Val Loss: 0.1360, Val F1(W): 0.9354
Epoch 15/100 - Train Loss: 0.1193, Val Loss: 0.1279, Val F1(W): 0.9395
Epoch 20/100 - Train Loss: 0.1113, Val Loss: 0.1224, Val F1(W): 0.9424
Epoch 25/100 - Train Loss: 0.1063, Val Loss: 0.1176, Val F1(W): 0.9424
Epoch 30/100 - Train Loss: 0.1037, Val Loss: 0.1125, Val F1(W): 0.9440

```

Epoch 35/100 - Train Loss: 0.1047, Val Loss: 0.1098, Val F1(W): 0.9452
Epoch 40/100 - Train Loss: 0.1014, Val Loss: 0.1069, Val F1(W): 0.9449
Epoch 45/100 - Train Loss: 0.0991, Val Loss: 0.1063, Val F1(W): 0.9455
Epoch 50/100 - Train Loss: 0.0980, Val Loss: 0.1042, Val F1(W): 0.9448
Epoch 55/100 - Train Loss: 0.0968, Val Loss: 0.1045, Val F1(W): 0.9437
Epoch 60/100 - Train Loss: 0.0943, Val Loss: 0.1011, Val F1(W): 0.9443
Epoch 65/100 - Train Loss: 0.0940, Val Loss: 0.1031, Val F1(W): 0.9427
Epoch 70/100 - Train Loss: 0.0942, Val Loss: 0.0992, Val F1(W): 0.9424
Epoch 75/100 - Train Loss: 0.0931, Val Loss: 0.0987, Val F1(W): 0.9426
Epoch 79: LR reduced from 0.000010 to 0.000007
Epoch 80/100 - Train Loss: 0.0924, Val Loss: 0.0987, Val F1(W): 0.9410
Epoch 85/100 - Train Loss: 0.0914, Val Loss: 0.0991, Val F1(W): 0.9371
Epoch 90/100 - Train Loss: 0.0907, Val Loss: 0.0990, Val F1(W): 0.9356
Epoch 92: LR reduced from 0.000007 to 0.000005
Epoch 95/100 - Train Loss: 0.0917, Val Loss: 0.0990, Val F1(W): 0.9346
Epoch 100/100 - Train Loss: 0.0896, Val Loss: 0.0971, Val F1(W): 0.9350

```

Training completed in 00:01:24 (hh:mm:ss) - 84.52 seconds

Completed. Training time: 00:01:24 (84.52s) - Final val loss: 0.0971

```

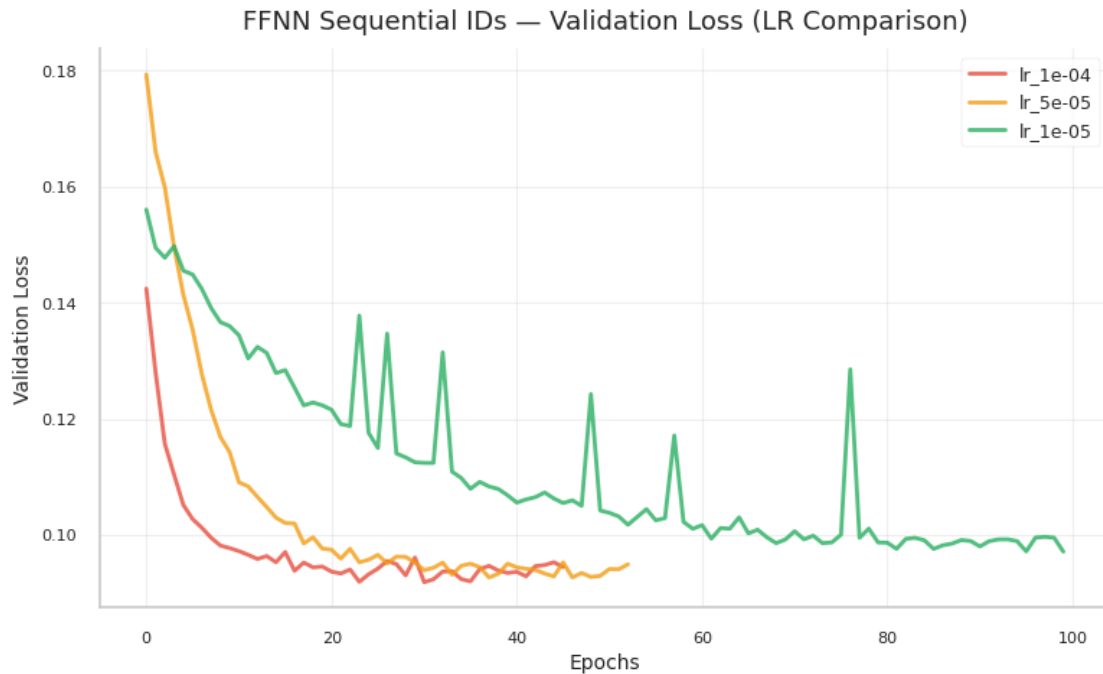
[ ]: # --- Plot loss curves ---

# Plot comparison
plot_lr_comparison(histories_seq, "FFNN Sequential IDs", save_dir)

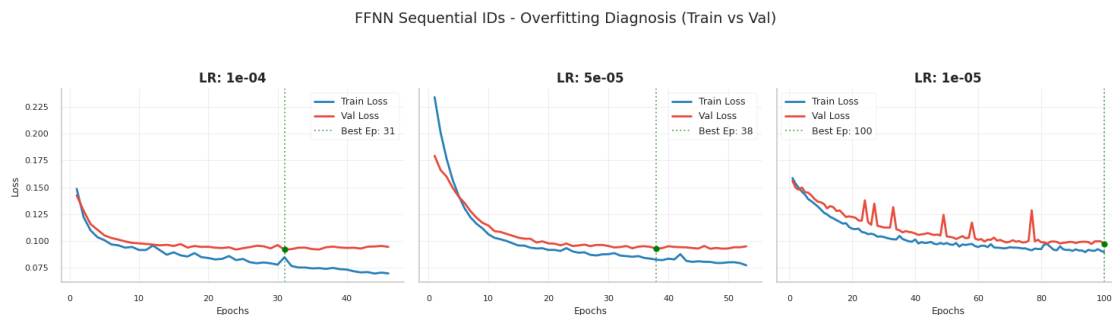
# Plot Train vs Validation
plot_train_val_grid(histories_seq, "FFNN Sequential IDs", save_dir)

```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task2_plots/ffnn_sequential_ids_lr_comparison.png



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task2_plots/ffnn_sequential_ids_diagnosis.png



```
[ ]: # --- Evaluate validation and test sets for ALL learning rates ---

print("\n" + "=" * 80)
print("Evaluation for ALL FFNN (Sequential IDs) learning rates")
print("=" * 80)

for lr in learning_rates_seq:
    model = models_seq[lr]

    print(f"\n{'='*60}")
```

```

print(f"FFNN (Sequential IDs) with lr={lr:.0e}")
print(f"{'='*60}")

# Validation evaluation
print(f'\nValidation report (FFNN Sequential IDs - lr={lr:.0e}):')
report = evaluate_model(model, X_val_tensor.to(device), y_val) # Moved
↪X_val_tensor to device
print(report)

# Test evaluation
print(f'Test report (FFNN Sequential IDs - lr={lr:.0e}):')
report_test = evaluate_model(model, X_test_tensor.to(device), y_test) #
↪Moved X_test_tensor to device
print(report_test)

# Find and highlight best model
best_lr_seq = min(histories_seq.items(), key=lambda x: min(x[1]['val_loss']))
print(f"\n{'='*60}")
print(f" Best FFNN (Sequential IDs) - LR: {best_lr_seq[0]:.0e} (Min Val Loss:
↪{min(best_lr_seq[1]['val_loss']):.4f})")
print(f"{'='*60}")

```

```

=====
Evaluation for ALL FFNN (Sequential IDs) learning rates
=====

```

```

=====
FFNN (Sequential IDs) with lr=1e-04
=====

```

Validation report (FFNN Sequential IDs - lr=1e-04):

	precision	recall	f1-score	support
0	0.1340	0.2295	0.1692	122
1	0.9692	0.9424	0.9556	3143
accuracy			0.9158	3265
macro avg	0.5516	0.5860	0.5624	3265
weighted avg	0.9380	0.9158	0.9263	3265

Test report (FFNN Sequential IDs - lr=1e-04):

	precision	recall	f1-score	support
0	0.1074	0.1193	0.1131	243
1	0.9657	0.9615	0.9636	6262

accuracy			0.9301	6505
macro avg	0.5365	0.5404	0.5383	6505
weighted avg	0.9336	0.9301	0.9318	6505

=====

FFNN (Sequential IDs) with lr=5e-05

=====

Validation report (FFNN Sequential IDs - lr=5e-05):

	precision	recall	f1-score	support
0	0.1250	0.1475	0.1353	122
1	0.9667	0.9599	0.9633	3143

accuracy			0.9296	3265
macro avg	0.5458	0.5537	0.5493	3265
weighted avg	0.9352	0.9296	0.9323	3265

Test report (FFNN Sequential IDs - lr=5e-05):

	precision	recall	f1-score	support
0	0.1461	0.1070	0.1235	243
1	0.9657	0.9757	0.9707	6262

accuracy			0.9433	6505
macro avg	0.5559	0.5414	0.5471	6505
weighted avg	0.9351	0.9433	0.9390	6505

=====

FFNN (Sequential IDs) with lr=1e-05

=====

Validation report (FFNN Sequential IDs - lr=1e-05):

	precision	recall	f1-score	support
0	0.0928	0.0738	0.0822	122
1	0.9643	0.9720	0.9682	3143

accuracy			0.9384	3265
macro avg	0.5286	0.5229	0.5252	3265
weighted avg	0.9318	0.9384	0.9350	3265

Test report (FFNN Sequential IDs - lr=1e-05):

	precision	recall	f1-score	support
0	0.1250	0.0864	0.1022	243

1	0.9650	0.9765	0.9707	6262
accuracy			0.9433	6505
macro avg	0.5450	0.5315	0.5365	6505
weighted avg	0.9336	0.9433	0.9383	6505

```
=====
Best FFNN (Sequential IDs) - LR: 1e-04 (Min Val Loss: 0.0919)
=====
```

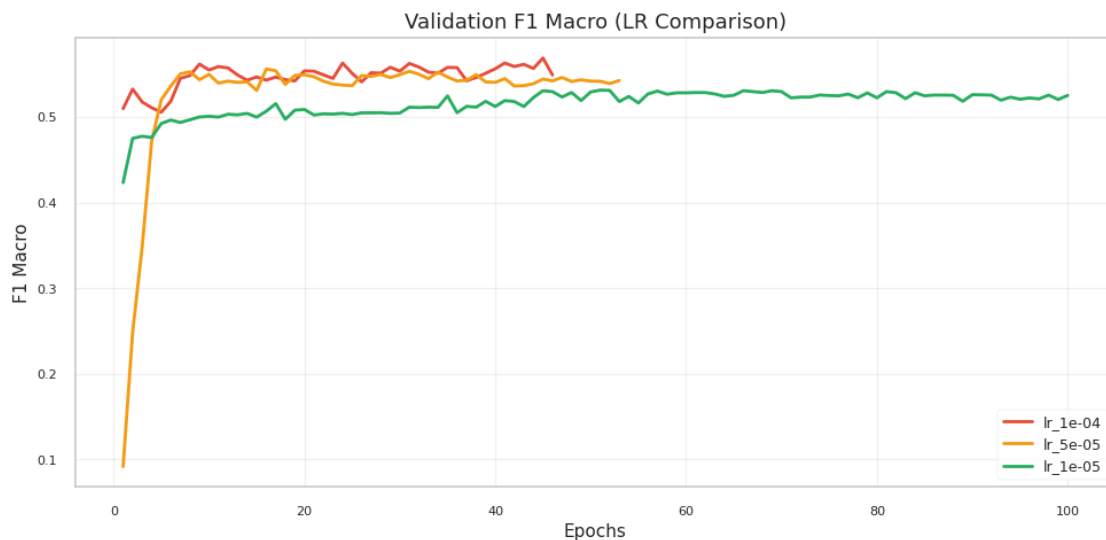
```
[ ]: # --- Plot metric curves for ALL learning rates (Sequential IDs) ---

print("\n" + "=" * 80)
print("Metric Curves for ALL FFNN (Sequential IDs) learning rates")
print("=" * 80)

plot_training_metrics(histories_seq, save_dir=save_dir,
    plot_name="ffnn_sequential_ids_f1_comparison")
```

```
=====
Metric Curves for ALL FFNN (Sequential IDs) learning rates
=====
```

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task2_plots/comparison_f1_macro.png



```
[ ]: # --- Plot confusion matrices for ALL learning rates (Sequential IDs) ---
    VALIDATION ---
```

```

print("\n" + "=" * 80)
print("Confusion matrices for ALL FFNN (Sequential IDs) learning rates -_
↳VALIDATION")
print("=" * 80)

for lr in learning_rates_seq:
    model = models_seq[lr]
    print(f"\nGenerating confusion matrix for FFNN (Sequential IDs) with lr={lr:
↳.0e}...")

    plot_cm_task2(model=model, X_tensor=X_val_tensor, y_tensor=y_val_tensor,
↳model_name=f"FFNN Sequential IDs (lr={lr:.0e})", save_dir=save_dir)

```

```

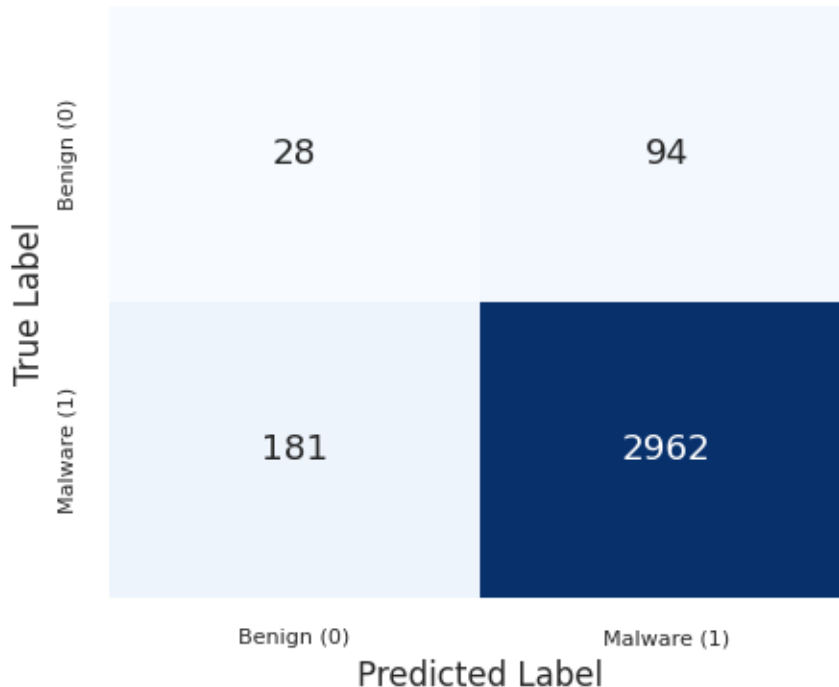
=====
Confusion matrices for ALL FFNN (Sequential IDs) learning rates - VALIDATION
=====

```

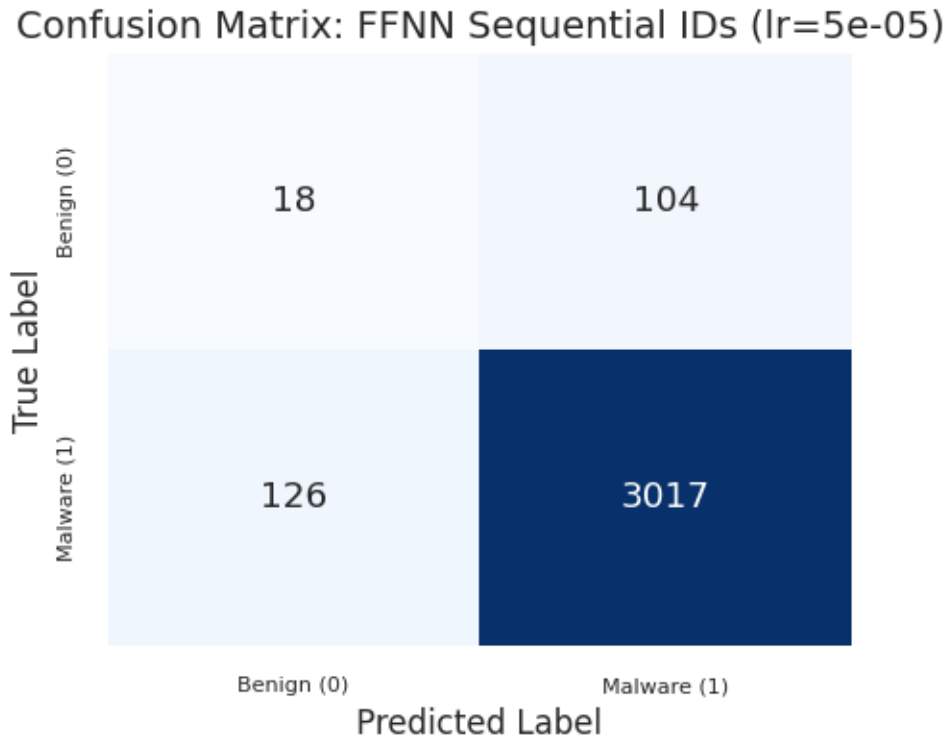
Generating confusion matrix for FFNN (Sequential IDs) with lr=1e-04...

Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task2_plots/cm_ffnn_sequential_ids_(lr=1e_04).png

Confusion Matrix: FFNN Sequential IDs (lr=1e-04)

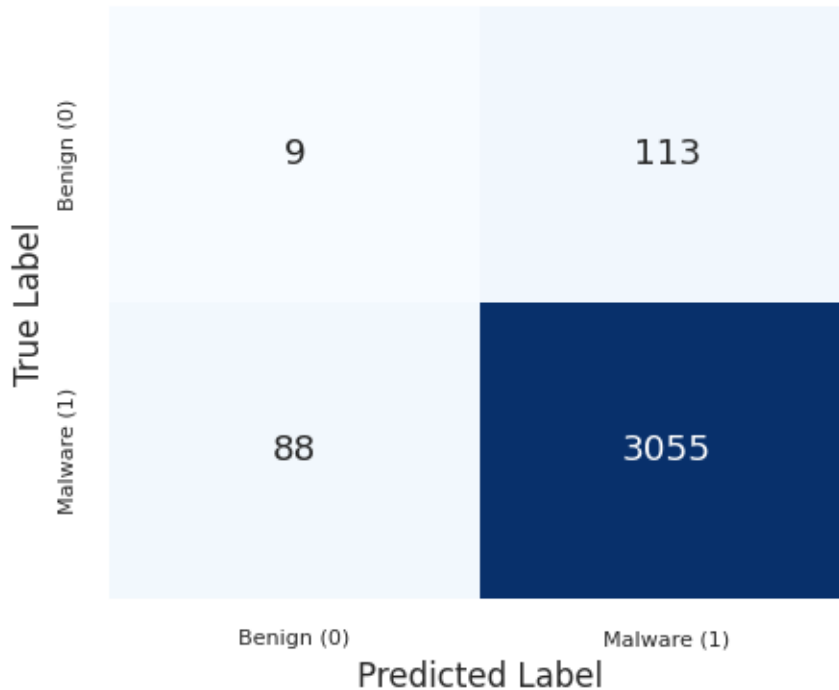


Generating confusion matrix for FFNN (Sequential IDs) with lr=5e-05...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_sequential_ids_(lr=5e_05).png



Generating confusion matrix for FFNN (Sequential IDs) with lr=1e-05...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_sequential_ids_(lr=1e_05).png

Confusion Matrix: FFNN Sequential IDs (lr=1e-05)



```
[ ]: # --- Plot confusion matrices for ALL learning rates (Sequential IDs) - TEST ---

print("\n" + "=" * 80)
print("Confusion matrices for ALL FFNN (Sequential IDs) learning rates - TEST")
print("=" * 80)

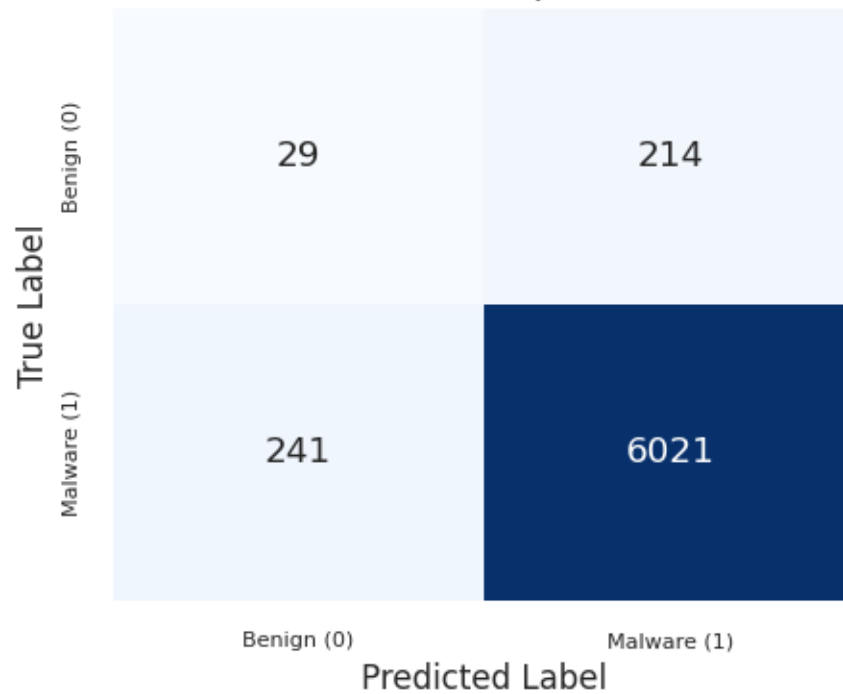
for lr in learning_rates_seq:
    model = models_seq[lr]
    print(f"\nGenerating confusion matrix for FFNN (Sequential IDs) with lr={lr:
↪.0e}...")

    plot_cm_task2(model=model, X_tensor=X_test_tensor, y_tensor=y_test_tensor,
↪model_name=f"FFNN Sequential IDs (lr={lr:.0e})", save_dir=save_dir)
```

```
=====
Confusion matrices for ALL FFNN (Sequential IDs) learning rates - TEST
=====

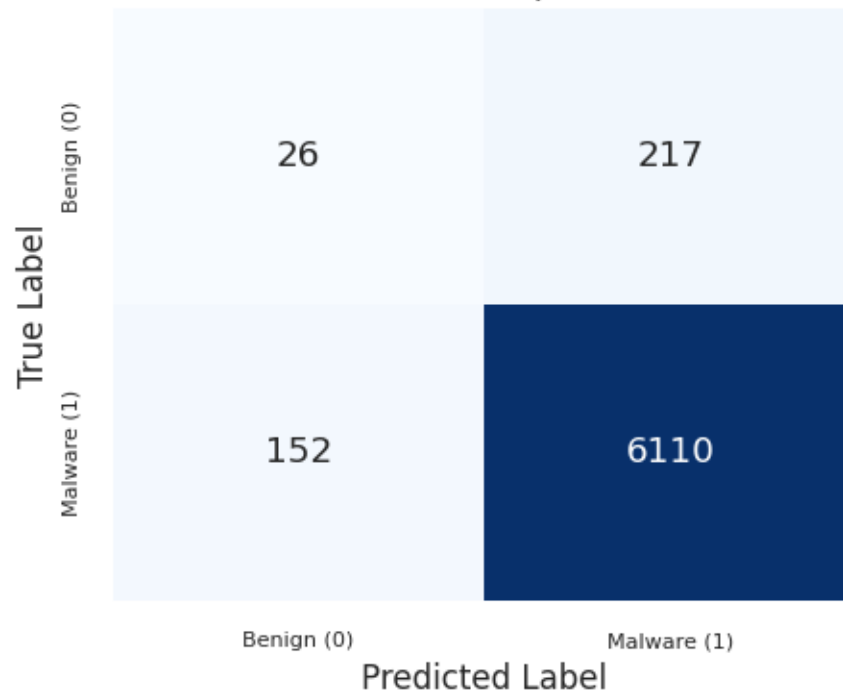
Generating confusion matrix for FFNN (Sequential IDs) with lr=1e-04...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_sequential_ids_(lr=1e_04).png
```

Confusion Matrix: FFNN Sequential IDs (lr=1e-04)



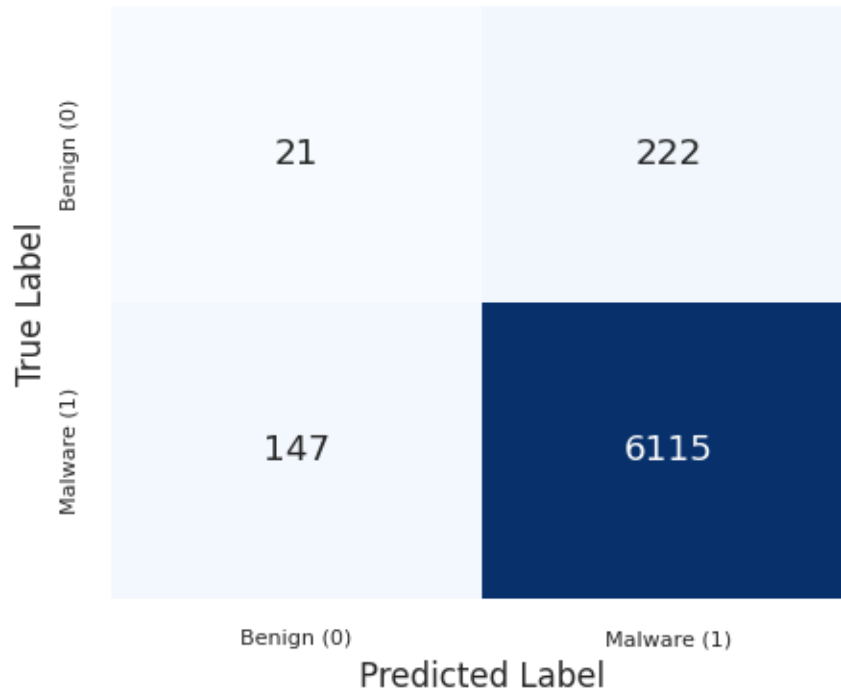
Generating confusion matrix for FFNN (Sequential IDs) with lr=5e-05..
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_sequential_ids_(lr=5e_05).png

Confusion Matrix: FFNN Sequential IDs (lr=5e-05)



Generating confusion matrix for FFNN (Sequential IDs) with lr=1e-05...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_sequential_ids_(lr=1e_05).png

Confusion Matrix: FFNN Sequential IDs (lr=1e-05)



Training with Learnable Embeddings Approach

```
[ ]: # --- Configuration Constants ---
seq_len = X_train.shape[1]
hidden_dims = [128, 64, 32]
dropout_rates = [0.4, 0.4, 0.4]
batchnorm_flags = [True, True, True]
min_delta = 0.00001
patience_early_stop = 15
epochs = 100

vocab_size = len(api_to_id)
embedding_dim = 128

# --- Device Setup (reuse from previous section if available) ---
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")

# Compute weights and move to device
pos_weight = compute_pos_weight(y_train_split).to(device)

# --- Define LR Candidates ---
learning_rates_emb = [0.00005, 0.00001, 0.000005]
```

```

histories_emb = {}      # Store results here
models_emb = {}         # Store models here
elapsed_emb_times = {} # Store elapsed times here

print(f"\n{'='*80}")
print("Training FFNN (Learnable Embeddings) with different learning rates")
print(f"{'='*80}")

for lr in learning_rates_emb:
    print(f"\n--- Training FFNN (Learnable Embeddings) with lr={lr:.0e} ---")

    # Initialize model with embedding layer and move to device
    model_emb = MalwareFFNN(
        seq_len,
        hidden_dims,
        dropout_rates,
        batchnorm_flags,
        use_embedding=True,
        vocab_size=vocab_size,
        embedding_dim=embedding_dim,
    ).to(device)

    # Criterion (pos_weight already on device)
    criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)

    # Initialize Optimizer
    optimizer = optim.AdamW(model_emb.parameters(), lr=lr, weight_decay=1e-4)

    # Setup Scheduler
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
        optimizer,
        mode='min',
        factor=0.5,
        patience=5,
        min_lr=1e-6
    )

    # Train the model
    model_emb, history, elapsed_emb = train_model(
        model_emb,
        train_loader,
        val_loader,
        epochs,
        optimizer,
        criterion,
        min_delta,
        patience_early_stop,

```

```

        scheduler=None
    )

    # Store model and history with numeric LR key
    models_emb[lr] = model_emb
    histories_emb[lr] = history
    elapsed_emb_times[lr] = elapsed_emb

    elapsed_str = time.strftime('%H:%M:%S', time.gmtime(elapsed_emb))
    print(f"Completed. Training time: {elapsed_str} ({elapsed_emb:.2f}s) -  

    ↪Final val loss: {history['val_loss'][-1]:.4f}")

```

Using device: cuda
Negatives: 489, Positives: 12571
pos_weight (applied to y=1 samples): 0.2000

=====
Training FFNN (Learnable Embeddings) with different learning rates
=====

--- Training FFNN (Learnable Embeddings) with lr=5e-05 ---

Epoch 1/100 - Train Loss: 0.1572, Val Loss: 0.1518, Val F1(W): 0.7537
Epoch 5/100 - Train Loss: 0.1177, Val Loss: 0.1227, Val F1(W): 0.9129
Epoch 10/100 - Train Loss: 0.0847, Val Loss: 0.0978, Val F1(W): 0.9297
Epoch 15/100 - Train Loss: 0.0626, Val Loss: 0.0794, Val F1(W): 0.9497
Epoch 20/100 - Train Loss: 0.0453, Val Loss: 0.0759, Val F1(W): 0.9469
Epoch 25/100 - Train Loss: 0.0351, Val Loss: 0.0786, Val F1(W): 0.9531
Epoch 30/100 - Train Loss: 0.0253, Val Loss: 0.0790, Val F1(W): 0.9570
Epoch 35/100 - Train Loss: 0.0207, Val Loss: 0.0858, Val F1(W): 0.9537
Early stopping at epoch 39 (best val loss: 0.072629)

Training completed in 00:00:38 (hh:mm:ss) - 38.24 seconds

Completed. Training time: 00:00:38 (38.24s) - Final val loss: 0.0989

--- Training FFNN (Learnable Embeddings) with lr=1e-05 ---

Epoch 1/100 - Train Loss: 0.1818, Val Loss: 0.1724, Val F1(W): 0.2773
Epoch 5/100 - Train Loss: 0.1663, Val Loss: 0.1608, Val F1(W): 0.5589
Epoch 10/100 - Train Loss: 0.1525, Val Loss: 0.1534, Val F1(W): 0.7078
Epoch 15/100 - Train Loss: 0.1397, Val Loss: 0.1410, Val F1(W): 0.8272
Epoch 20/100 - Train Loss: 0.1277, Val Loss: 0.1374, Val F1(W): 0.8360
Epoch 25/100 - Train Loss: 0.1169, Val Loss: 0.1292, Val F1(W): 0.8755
Epoch 30/100 - Train Loss: 0.1084, Val Loss: 0.1220, Val F1(W): 0.8878
Epoch 35/100 - Train Loss: 0.1000, Val Loss: 0.1144, Val F1(W): 0.9164
Epoch 40/100 - Train Loss: 0.0912, Val Loss: 0.1063, Val F1(W): 0.9199
Epoch 45/100 - Train Loss: 0.0843, Val Loss: 0.1022, Val F1(W): 0.9241
Epoch 50/100 - Train Loss: 0.0779, Val Loss: 0.0965, Val F1(W): 0.9297
Epoch 55/100 - Train Loss: 0.0718, Val Loss: 0.0934, Val F1(W): 0.9353
Epoch 60/100 - Train Loss: 0.0663, Val Loss: 0.0865, Val F1(W): 0.9384

```
Epoch 65/100 - Train Loss: 0.0613, Val Loss: 0.0831, Val F1(W): 0.9408
Epoch 70/100 - Train Loss: 0.0562, Val Loss: 0.0797, Val F1(W): 0.9427
Epoch 75/100 - Train Loss: 0.0517, Val Loss: 0.0779, Val F1(W): 0.9435
Epoch 80/100 - Train Loss: 0.0463, Val Loss: 0.0764, Val F1(W): 0.9427
Epoch 85/100 - Train Loss: 0.0438, Val Loss: 0.0742, Val F1(W): 0.9476
Epoch 90/100 - Train Loss: 0.0395, Val Loss: 0.0751, Val F1(W): 0.9421
Epoch 95/100 - Train Loss: 0.0389, Val Loss: 0.0739, Val F1(W): 0.9482
Epoch 100/100 - Train Loss: 0.0349, Val Loss: 0.0744, Val F1(W): 0.9473
```

```
Training completed in 00:01:36 (hh:mm:ss) - 96.93 seconds
Completed. Training time: 00:01:36 (96.93s) - Final val loss: 0.0744
```

```
--- Training FFNN (Learnable Embeddings) with lr=5e-06 ---
```

```
Epoch 1/100 - Train Loss: 0.1960, Val Loss: 0.1750, Val F1(W): 0.2801
Epoch 5/100 - Train Loss: 0.1865, Val Loss: 0.1715, Val F1(W): 0.3742
Epoch 10/100 - Train Loss: 0.1745, Val Loss: 0.1673, Val F1(W): 0.4756
Epoch 15/100 - Train Loss: 0.1644, Val Loss: 0.1628, Val F1(W): 0.5683
Epoch 20/100 - Train Loss: 0.1557, Val Loss: 0.1570, Val F1(W): 0.6910
Epoch 25/100 - Train Loss: 0.1473, Val Loss: 0.1527, Val F1(W): 0.7267
Epoch 30/100 - Train Loss: 0.1421, Val Loss: 0.1448, Val F1(W): 0.7968
Epoch 35/100 - Train Loss: 0.1356, Val Loss: 0.1436, Val F1(W): 0.7968
Epoch 40/100 - Train Loss: 0.1290, Val Loss: 0.1424, Val F1(W): 0.8143
Epoch 45/100 - Train Loss: 0.1246, Val Loss: 0.1334, Val F1(W): 0.8527
Epoch 50/100 - Train Loss: 0.1193, Val Loss: 0.1343, Val F1(W): 0.8705
Epoch 55/100 - Train Loss: 0.1134, Val Loss: 0.1238, Val F1(W): 0.8897
Epoch 60/100 - Train Loss: 0.1084, Val Loss: 0.1204, Val F1(W): 0.8912
Epoch 65/100 - Train Loss: 0.1043, Val Loss: 0.1194, Val F1(W): 0.8844
Epoch 70/100 - Train Loss: 0.0997, Val Loss: 0.1160, Val F1(W): 0.8993
Epoch 75/100 - Train Loss: 0.0959, Val Loss: 0.1160, Val F1(W): 0.8961
Epoch 80/100 - Train Loss: 0.0910, Val Loss: 0.1088, Val F1(W): 0.9097
Epoch 85/100 - Train Loss: 0.0868, Val Loss: 0.1076, Val F1(W): 0.9144
Epoch 90/100 - Train Loss: 0.0834, Val Loss: 0.1027, Val F1(W): 0.9163
Epoch 95/100 - Train Loss: 0.0805, Val Loss: 0.1014, Val F1(W): 0.9244
Epoch 100/100 - Train Loss: 0.0781, Val Loss: 0.0988, Val F1(W): 0.9242
```

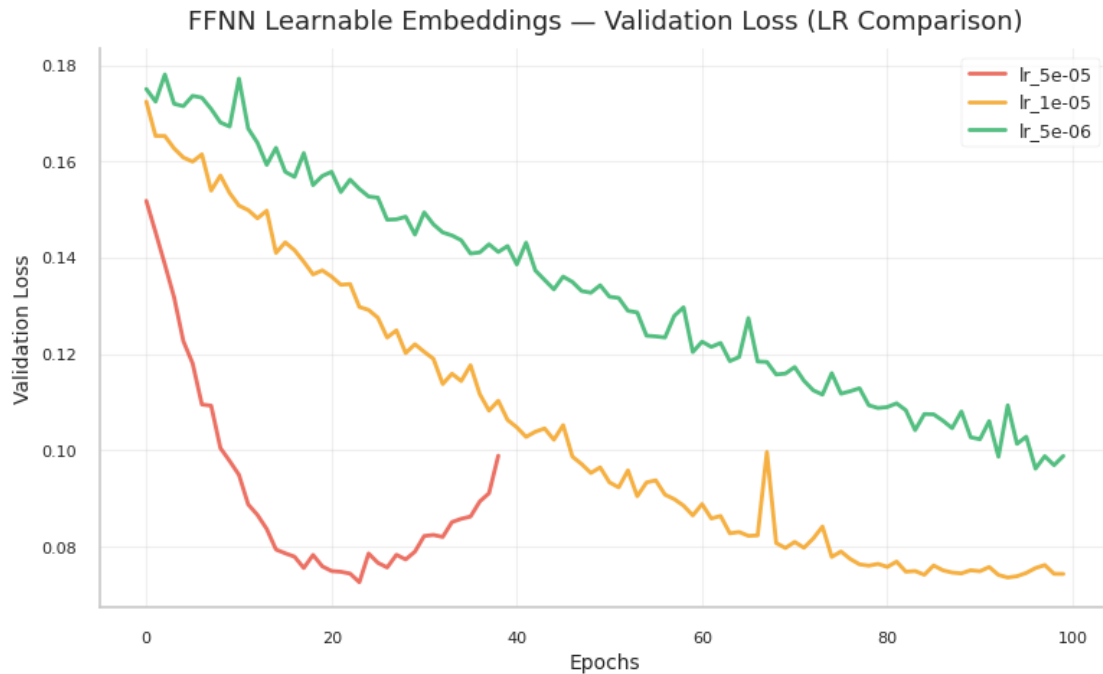
```
Training completed in 00:01:36 (hh:mm:ss) - 96.88 seconds
Completed. Training time: 00:01:36 (96.88s) - Final val loss: 0.0988
```

```
[ ]: # --- Plot loss curves ---

# Plot comparison
plot_lr_comparison(histories_emb, "FFNN Learnable Embeddings", save_dir)

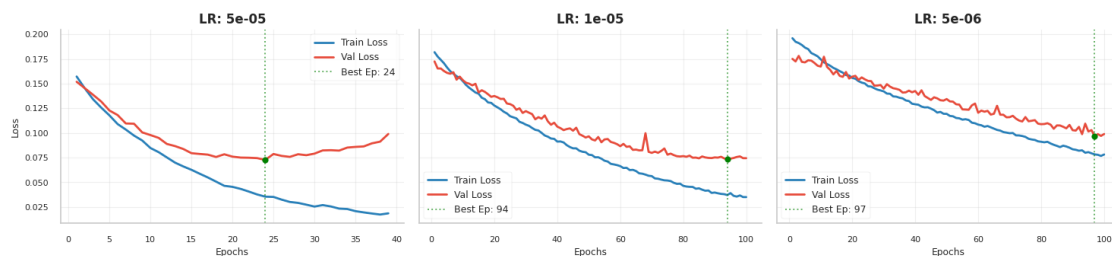
# Plot Train vs Validation
plot_train_val_grid(histories_emb, "FFNN Learnable Embeddings", save_dir)
```

```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/ffnn_learnable_embeddings_lr_comparison.png
```



Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task2_plots/ffnn_learnable_embeddings_diagnosis.png

FFNN Learnable Embeddings - Overfitting Diagnosis (Train vs Val)



```
[ ]: # --- Evaluate validation and test sets for ALL learning rates ---

print("\n" + "=" * 80)
print("Evaluation for ALL FFNN (Learnable Embeddings) learning rates")
print("=" * 80)

for lr in learning_rates_emb:
    model = models_emb[lr]

    print(f"\n{'='*60}")
```



```

print(f"FFNN (Learnable Embeddings) with lr={lr:.0e}")
print(f"{'='*60}")

# Validation evaluation
print(f'\nValidation report (FFNN Learnable Embeddings - lr={lr:.0e}):')
report = evaluate_model(model, X_val_tensor.to(device), y_val) # Moved
↪X_val_tensor to device
print(report)

# Test evaluation
print(f'Test report (FFNN Learnable Embeddings - lr={lr:.0e}):')
report_test = evaluate_model(model, X_test_tensor.to(device), y_test) #
↪Moved X_test_tensor to device
print(report_test)

# Find and highlight best model
best_lr_emb = min(histories_emb.items(), key=lambda x: min(x[1]['val_loss']))
print(f"\n{'='*60}")
print(f" Best FFNN (Learnable Embeddings) - LR: {best_lr_emb[0]:.0e} (Min Val
↪Loss: {min(best_lr_emb[1]['val_loss']):.4f})")
print(f"{'='*60}")

```

```

=====
Evaluation for ALL FFNN (Learnable Embeddings) learning rates
=====

```

```

=====
FFNN (Learnable Embeddings) with lr=5e-05
=====

```

Validation report (FFNN Learnable Embeddings - lr=5e-05):

	precision	recall	f1-score	support
0	0.3267	0.4016	0.3603	122
1	0.9766	0.9679	0.9722	3143
accuracy			0.9467	3265
macro avg	0.6516	0.6848	0.6662	3265
weighted avg	0.9523	0.9467	0.9493	3265

Test report (FFNN Learnable Embeddings - lr=5e-05):

	precision	recall	f1-score	support
0	0.1712	0.3086	0.2203	243
1	0.9723	0.9420	0.9569	6262

accuracy			0.9184	6505
macro avg	0.5718	0.6253	0.5886	6505
weighted avg	0.9424	0.9184	0.9294	6505

```
=====
FFNN (Learnable Embeddings) with lr=1e-05
=====
```

Validation report (FFNN Learnable Embeddings - lr=1e-05):

	precision	recall	f1-score	support
0	0.3385	0.3607	0.3492	122
1	0.9751	0.9726	0.9739	3143

accuracy			0.9498	3265
macro avg	0.6568	0.6666	0.6615	3265
weighted avg	0.9513	0.9498	0.9505	3265

Test report (FFNN Learnable Embeddings - lr=1e-05):

	precision	recall	f1-score	support
0	0.1896	0.2099	0.1992	243
1	0.9692	0.9652	0.9672	6262

accuracy			0.9370	6505
macro avg	0.5794	0.5875	0.5832	6505
weighted avg	0.9401	0.9370	0.9385	6505

```
=====
FFNN (Learnable Embeddings) with lr=5e-06
=====
```

Validation report (FFNN Learnable Embeddings - lr=5e-06):

	precision	recall	f1-score	support
0	0.2468	0.4754	0.3249	122
1	0.9789	0.9437	0.9610	3143

accuracy			0.9262	3265
macro avg	0.6128	0.7095	0.6429	3265
weighted avg	0.9515	0.9262	0.9372	3265

Test report (FFNN Learnable Embeddings - lr=5e-06):

	precision	recall	f1-score	support
0	0.1490	0.3580	0.2104	243

1	0.9737	0.9206	0.9464	6262
accuracy			0.8996	6505
macro avg	0.5613	0.6393	0.5784	6505
weighted avg	0.9428	0.8996	0.9189	6505

```
=====
Best FFNN (Learnable Embeddings) - LR: 5e-05 (Min Val Loss: 0.0726)
=====
```

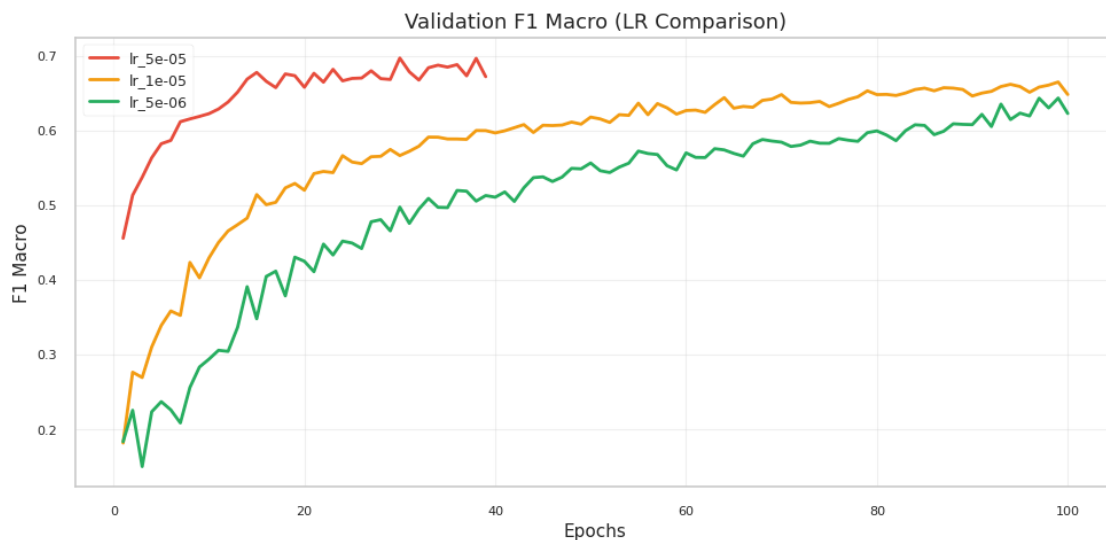
```
[ ]: # --- Plot metric curves for ALL learning rates (Learnable Embeddings) ---

print("\n" + "=" * 80)
print("Metric Curves for ALL FFNN (Learnable Embeddings) learning rates")
print("=" * 80)

# The plot_training_metrics function is designed to compare multiple histories.
plot_training_metrics(histories_emb, save_dir=save_dir,
    plot_name="ffnn_learnable_embeddings_f1_comparison")
```

```
=====
Metric Curves for ALL FFNN (Learnable Embeddings) learning rates
=====
```

```
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/comparison_f1_macro.png
```



```
[ ]: # --- Plot confusion matrices for ALL learning rates (Learnable Embeddings) - VALIDATION ---

print("\n" + "=" * 80)
print("Confusion matrices for ALL FFNN (Learnable Embeddings) learning rates - VALIDATION")
print("=" * 80)

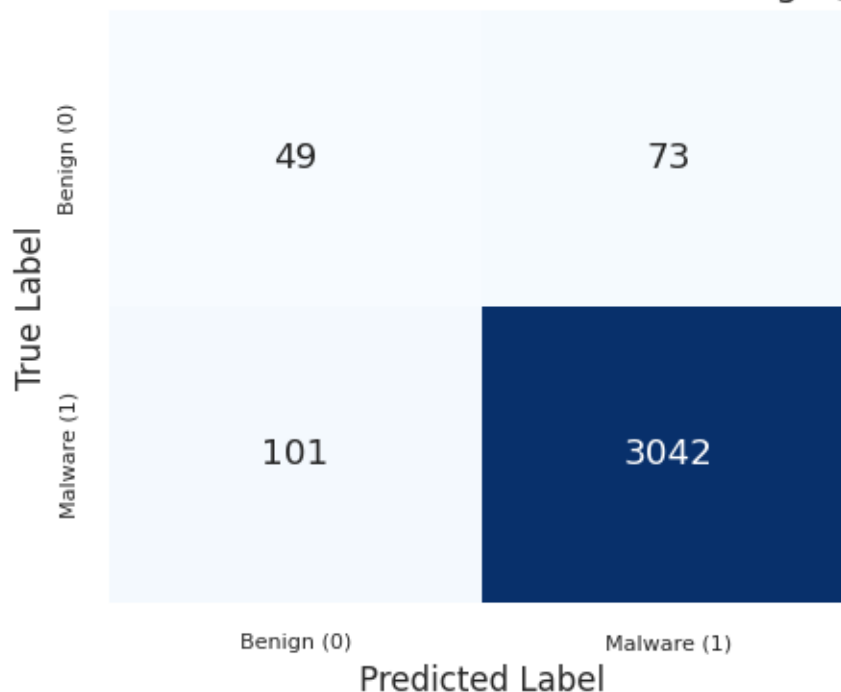
for lr in learning_rates_emb:
    model = models_emb[lr]
    print(f"\nGenerating confusion matrix for FFNN (Learnable Embeddings) with lr={lr:.0e}...")

    plot_cm_task2(model=model, X_tensor=X_val_tensor, y_tensor=y_val_tensor, model_name=f"FFNN Learnable Embeddings (lr={lr:.0e})", save_dir=save_dir)
```

```
=====
Confusion matrices for ALL FFNN (Learnable Embeddings) learning rates -
VALIDATION
=====
```

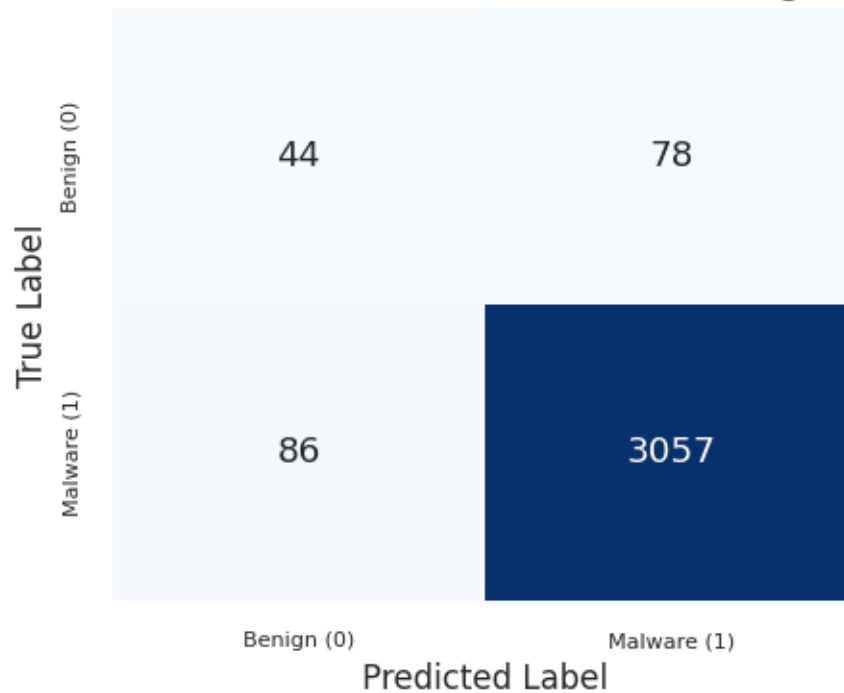
```
Generating confusion matrix for FFNN (Learnable Embeddings) with lr=5e-05...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_learnable_embeddings_(lr=5e_05).png
```

Confusion Matrix: FFNN Learnable Embeddings (lr=5e-05)



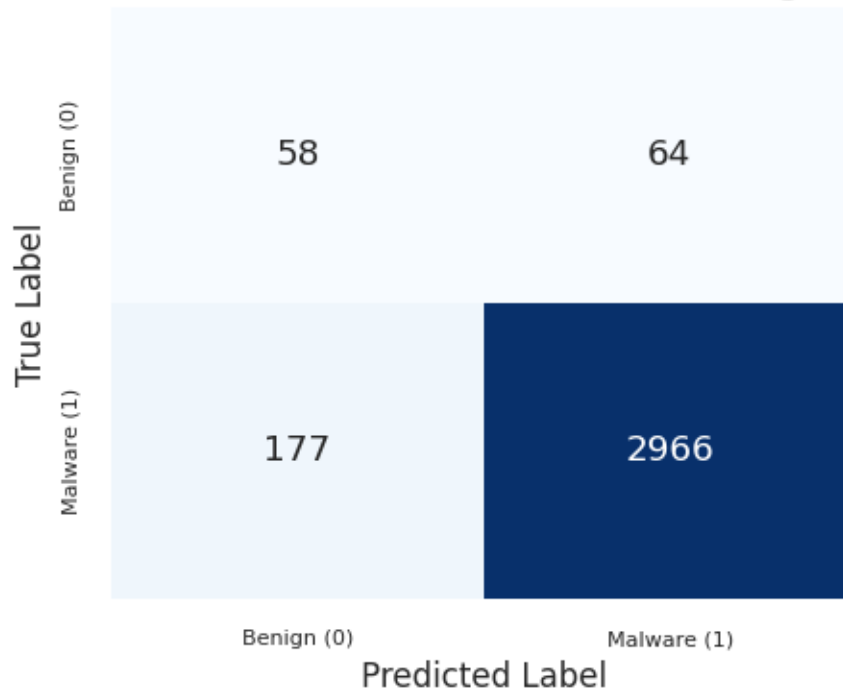
Generating confusion matrix for FFNN (Learnable Embeddings) with $lr=1e-05$...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task2_plots/cm_ffnn_learnable_embeddings_(lr=1e_05).png

Confusion Matrix: FFNN Learnable Embeddings ($lr=1e-05$)



Generating confusion matrix for FFNN (Learnable Embeddings) with $lr=5e-06$...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images/task2_plots/cm_ffnn_learnable_embeddings_(lr=5e_06).png

Confusion Matrix: FFNN Learnable Embeddings (lr=5e-06)



```
[ ]: # --- Plot confusion matrices for ALL learning rates (Learnable Embeddings) - TEST ---

print("\n" + "=" * 80)
print("Confusion matrices for ALL FFNN (Learnable Embeddings) learning rates - TEST")
print("=" * 80)

for lr in learning_rates_emb:
    model = models_emb[lr]
    print(f"\nGenerating confusion matrix for FFNN (Learnable Embeddings) with lr={lr:.0e}...")

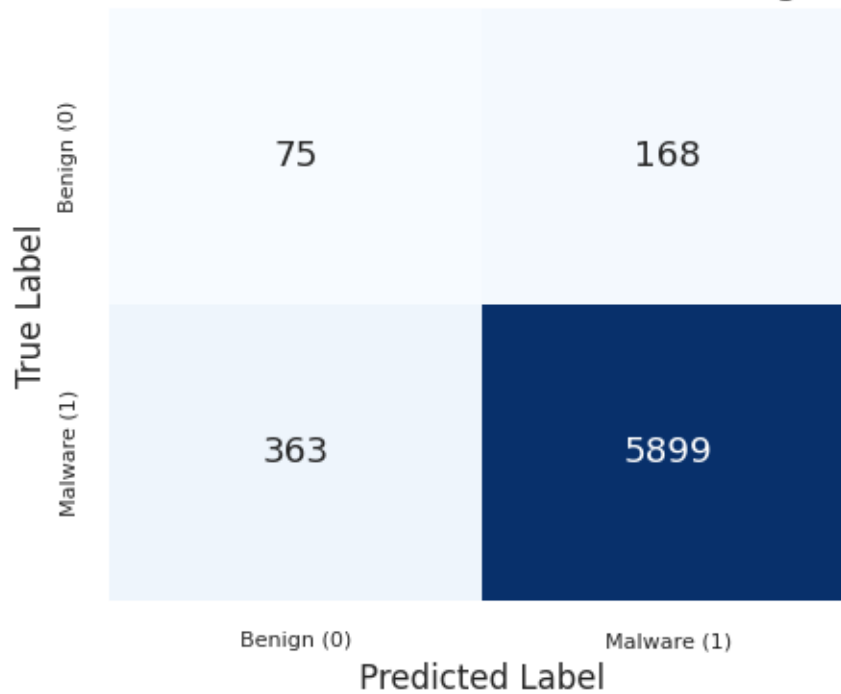
    plot_cm_task2(model=model, X_tensor=X_test_tensor, y_tensor=y_test_tensor, model_name=f"FFNN Learnable Embeddings (lr={lr:.0e})", save_dir=save_dir)
```

=====
Confusion matrices for ALL FFNN (Learnable Embeddings) learning rates - TEST
=====

Generating confusion matrix for FFNN (Learnable Embeddings) with lr=5e-05...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images

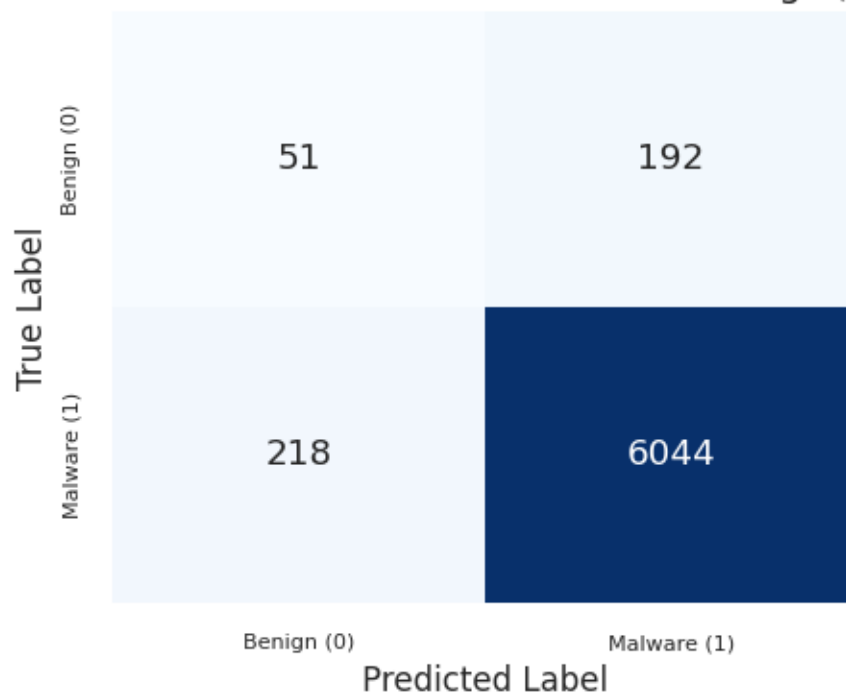
/task2_plots/cm_ffnn_learnable_embeddings_(lr=5e_05).png

Confusion Matrix: FFNN Learnable Embeddings (lr=5e-05)



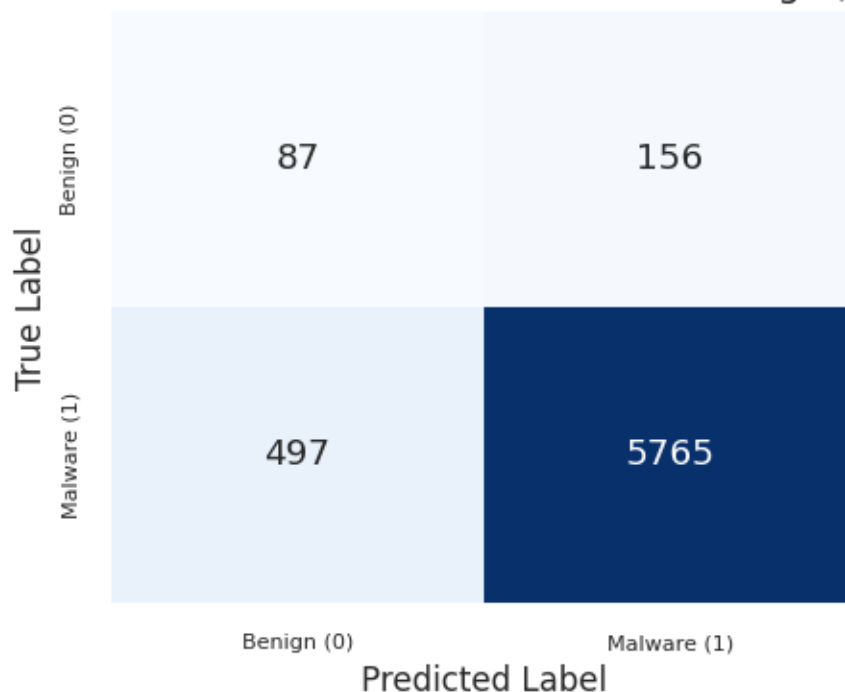
Generating confusion matrix for FFNN (Learnable Embeddings) with lr=1e-05...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_learnable_embeddings_(lr=1e_05).png

Confusion Matrix: FFNN Learnable Embeddings (lr=1e-05)



Generating confusion matrix for FFNN (Learnable Embeddings) with lr=5e-06...
Saved plot: /content/drive/MyDrive/Projects/AImSecure/Laboratory2/results/images
/task2_plots/cm_ffnn_learnable_embeddings_(lr=5e_06).png

Confusion Matrix: FFNN Learnable Embeddings ($\text{lr}=5\text{e-}06$)



Q: Use a FFNN in both cases. Report how you selected the hyperparameters of your final model, and justify your choices. For Task 2, we implemented two distinct FFNN configurations: one using Sequential Identifiers and another utilizing Learnable Embeddings. The hyperparameter selection was driven by the need to handle a severely imbalanced dataset while maintaining architectural flexibility.

1. **Architecture and Regularization Hidden Layers:** We adopted a funnel-shaped architecture. For the Sequential IDs model, we used [256, 128, 64], while for the Embedding model, we used [128, 64, 32]. This design forces the network to compress information and learn increasingly abstract representations of API patterns.

Batch Normalization & Dropout: Every hidden layer includes Batch Normalization to stabilize training and accelerate convergence. We applied a high Dropout rate (up to 0.4) to mitigate overfitting, which is a significant risk given the small number of benign samples compared to the majority class. Then, as we move to the next layers, the dropout layer decreased initially to 0.35, and at the end to 0.3

2. **Addressing Class Imbalance Weighted Loss:** We used BCEWithLogitsLoss with a `pos_weight` of 0.20 (calculated as the ratio of benign to malware samples). This penalizes errors on the minority class more heavily, directly aiming to improve the Recall for benign software, which is critical in a cybersecurity context.
3. **Training Strategy Optimization:** We used AdamW with `weight_decay` ($1\text{e-}4$ for Learnable Embeddings and $1\text{e-}5$ for Sequential IDs) for effective weight regularization.

Learning Rate (LR) Selection: Through an LR grid search:

For Sequential IDs, 1e-04 proved to be the most stable.

For Learnable Embeddings, 5e-05 yielded the best performance, achieving the lowest minimum validation loss (0.0726).

Early Stopping: We implemented early stopping with a patience of 15 epochs based on validation loss to prevent the model from memorizing training noise.

Q: Can you obtain the same results for sequential identifiers and learnable embeddings? If not, why? No, the results demonstrate that the Learnable Embeddings approach significantly outperforms the Sequential Identifiers.

1. Performance Comparison (Macro F1-score) Sequential IDs: The best model (LR 1e-04) achieved a Macro F1-score of 0.56 in validation and 0.53 in test. The Recall for the benign class was extremely low (ranging between 0.11 and 0.22), showing that the model struggled to identify benign samples.

Learnable Embeddings: The best model (LR 5e-05) significantly improved these metrics, reaching a Macro F1-score of 0.66 in validation and 0.58 in test. Crucially, the benign Recall increased to 0.40 in the validation set.

2. Technical Justification: Why Embeddings are Superior Semantic Representation: Sequential identifiers treat API calls as ordinal numbers, implying a mathematical relationship (e.g., API 5 is “closer” to 6 than to 10) that does not exist. Embeddings, however, map each API into a continuous vector space where the network can learn semantic proximity. API calls with similar functional roles (e.g., registry modifications) cluster together, providing richer features.

Feature Enrichment: The embedding layer (dim=128) transforms a single integer ID into a high-dimensional representation. This allows the FFNN to capture complex interactions and hidden dependencies between API calls that raw identifiers simply cannot represent.

Generalization: The training curves show that the Embedding model reaches a much lower validation loss (0.07 vs 0.09). While the FFNN architecture still suffers from ignoring the strict temporal order of calls (addressed later with RNNs), the learnable embeddings provide a much more discriminative input for the classifier compared to raw IDs.

```
[ ]: # =====  
# FINAL SUMMARY  
# =====  
  
# Helper function to extract macro avg from the best FFNN model  
def get_macro_avg_stats_ffnn(model, X_tensor):  
    """  
    Predict and get macro average statistics for an FFNN model.  
    """  
    # Get the device from the model itself  
    device = next(model.parameters()).device
```

```

model.eval()
with torch.no_grad():
    outputs = model(X_tensor.to(device)) # Move input tensor to device
    probs = torch.sigmoid(outputs)
    y_pred = (probs > 0.5).long().cpu().numpy().flatten()

y_true = y_val # Using validation set
report_dict = classification_report(y_true, y_pred, output_dict=True,
    ↪ zero_division=0)
return report_dict['macro avg']

# --- 1. Prepare Stats for FFNN Sequential IDs ---
best_model_seq_obj = models_seq[best_lr_seq[0]]
macro_seq = get_macro_avg_stats_ffnn(best_model_seq_obj, X_val_tensor)

# --- 2. Prepare Stats for FFNN Learnable Embeddings ---
best_model_emb_obj = models_emb[best_lr_emb[0]]
macro_emb = get_macro_avg_stats_ffnn(best_model_emb_obj, X_val_tensor)

# --- PRINTING THE REPORT ---

print("\n" + "=" * 100)
print("TASK 2 SUMMARY: Best Learning Rates & Macro Avg Performance")
print("=" * 100)
# Header for the manual table
print(f"{'Model':<30} | {'Best LR':<10} | {'Val Loss':<10} | {'Macro P':<8} | "
    ↪ {'Macro R':<8} {'Macro F1':<8} | {'Time':<10}")
print("-" * 100)

# 1. FFNN Sequential IDs
t_seq = time.strftime('%H:%M:%S', time.
    ↪ gmtime(elapsed_seq_times[best_lr_seq[0]]))
print(f"{'FFNN (Sequential IDs)':<30} | {best_lr_seq[0]:.0e} | "
    ↪ {min(best_lr_seq[1]['val_loss']):.4f} | "
        f"{macro_seq['precision']:.4f} {macro_seq['recall']:.4f} "
    ↪ {macro_seq['f1-score']:.4f} | {t_seq}")

# 2. FFNN Learnable Embeddings
t_emb = time.strftime('%H:%M:%S', time.
    ↪ gmtime(elapsed_emb_times[best_lr_emb[0]]))
print(f"{'FFNN (Learnable Embeddings)':<30} | {best_lr_emb[0]:.0e} | "
    ↪ {min(best_lr_emb[1]['val_loss']):.4f} | "
        f"{macro_emb['precision']:.4f} {macro_emb['recall']:.4f} "
    ↪ {macro_emb['f1-score']:.4f} | {t_emb}")

```

=====